

PiCamera V1.13 操作手册

PiCamera

该软件包为 Python 2.7（或更高版本）或 Python 3.2（或更高版本）的 Raspberry Pi 相机模块提供了纯 Python 接口。

Link

- 该代码在 BSD 许可下获得许可
- GitHub上托管了picamera包的源代码，并且也托管了错误跟踪器
- 可以在ReadTheDocs上阅读文档（包括安装、快速入门示例和大量代码配方）
- 包可以从 PyPI 下载，但阅读安装说明更有可能有用

目录

1. 安装
2. 入门
3. 基本操作
4. 进阶操作
5. 常见问题与解答 (FAQ)
6. 相机硬件
7. 开发
8. 弃用功能
9. 应用程序接口 (API) — PiCamera
10. 应用程序接口 (API) — 视频流
11. 应用程序接口 (API) — 渲染器
12. 应用程序接口 (API) — 编码器
13. 应用程序接口 (API) — 异常
14. 应用程序接口 (API) — 颜色与配色
15. 应用程序接口 (API) — 数组
16. 应用程序接口 (API) — MMAL
17. 更改日志
18. 许可证

参数和表格

- 参量
- 模块索引
- 搜索页面

一、安装

1.1 树莓派系统安装

如果您使用的是 Raspbian 发行版，那么您可能默认安装了 picamera。您可以通过启动 Python 并尝试导入 picamera 来测试调用：

```
$ python -c "import picamera"
$ python3 -c "import picamera"
```

如果没有出现任何错误，那么说明您已经安装了 picamera 软件包，可以继续下一步的工作了。如果像下面那样，您还没有安装 picamera 软件包：

```
$ python -c "import picamera"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
ImportError: No module named picamera
$ python3 -c "import picamera"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
ImportError: No module named 'picamera'
```

要在 Raspbian 上安装 picamera，最好使用系统的包管理器：apt。这将确保 picamera 易于保持最新状态，并且如果您愿意，也可以轻松删除。它还将使系统上的所有用户都可以使用 picamera。要使用 apt 安装 picamera，只需运行：

```
$ sudo apt-get update
$ sudo apt-get install python-picamera python3-picamera
```

要在发布新版本时升级您的安装，您只需使用 apt 的正常升级程序：

```
$ sudo apt-get update
$ sudo apt-get upgrade
```

如果您想卸载 picamera：

```
$ sudo apt-get remove python-picamera python3-picamera
```

1.2 在其他Linux系统下的安装

在 Raspbian 以外的发行版上，使用 Python 的 pip 工具在系统范围内安装可能是最简单的：

```
$ sudo pip install picamera
```

如果你想使用 `picamera.array` 模块中的类，那么指定“array”选项，它依赖于numpy：

```
$ sudo pip install "picamera[array]"
```

Warning

请注意，旧版本的 pip 将尝试从源代码构建 numpy。这将在 Pi 上花费很长时间（在较慢的模型上需要几个小时）。新版本的 pip 将下载并安装一个预先构建的 numpy “轮子（wheel）”，因此更快。

要升级picamera的版本，您仅需：

```
$ sudo pip install -U picamera
```

要卸载picamera，您仅需：

```
$ sudo pip uninstall picamera
```

1.3 固件升级

树莓派的相机模块的行为由其固件决定。随着新版本的发布，树莓派相机模块也进行了不断的错误修复和新的固件版本扩展。虽然 `picamera` 库试图保持与旧的树莓派相机固件的向下兼容，但它仅在发布时针对最新固件进行了测试，如果您运行的是旧固件，则并非所有功能都可用。例如，`annotate_text` 属性依赖于最新的固件；旧固件缺乏该功能。

您可以使用以下命令确定当前固件的版本：

```
uname -a
```

固件版本在#后显示：

```
Linux kermi 3.12.26+ #707 PREEMPT Sat Aug 30 17:39:19 BST 2014
armv6l GNU/Linux
firmware revision --+
```

在树莓派官方系统中，标准升级程序可以使您的固件保持最新：

```
$ sudo apt-get update
$ sudo apt-get upgrade
```

Warning

之前的文档建议使用 `rpi-update` 实用程序来更新树莓派的固件；现在不建议进行这样的操作。如果您之前使用过 `rpi-update` 实用程序来更新您的固件，您可以使用以下命令切换回使用 `apt` 来管理它：

```
$ sudo apt-get update
$ sudo apt-get install --reinstall libraspberrypi0
libraspberrypi-{bin,dev,doc} z
> raspberrypi-bootloader
$ sudo rm /boot/.firmware_revision
```

更新完成后，您需要重启树莓派。

Note

树莓派的TFT 屏幕（和类似的 GPIO 驱动屏幕）需要自定义固件才能运行。该固件版本落后于官方固件版本，并且缺少包括长时间曝光、文本叠加的多项功能。

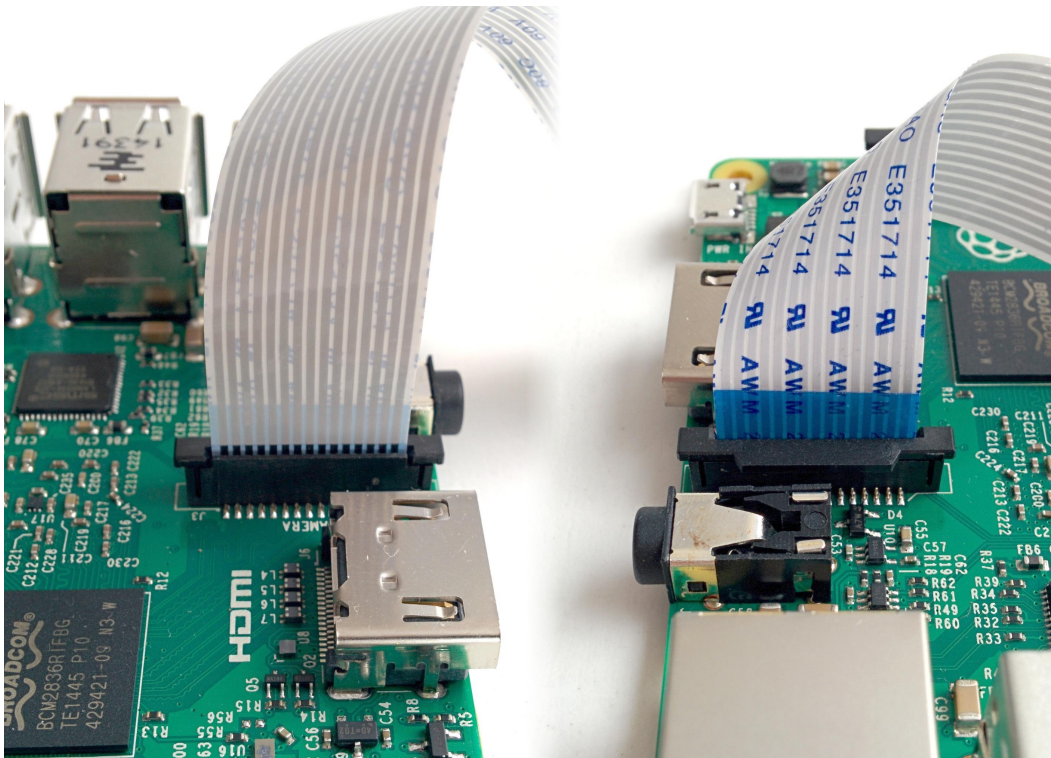
二、入门

Warning

安装摄像头模块时，请确保您的树莓派处于关闭状态。尽管可以在树莓派开启时安装摄像头，但这不是一个好习惯（如果摄像头在移除时处于活动状态，则可能会损坏它）。

将相机模块连接到树莓派上的 CSI（Camera Serial Interface）端口；这是与 HDMI 插座相邻的细长端口。轻轻提起 CSI 端口顶部的项圈（如果它脱落，请不要担心，您可以将其推回，但以后尽量轻柔！）。将摄像头模块的带状电缆滑入端口，蓝色面向以太网端口（或者如果您有 A/A+ 型，以太网端口所在的位置）。

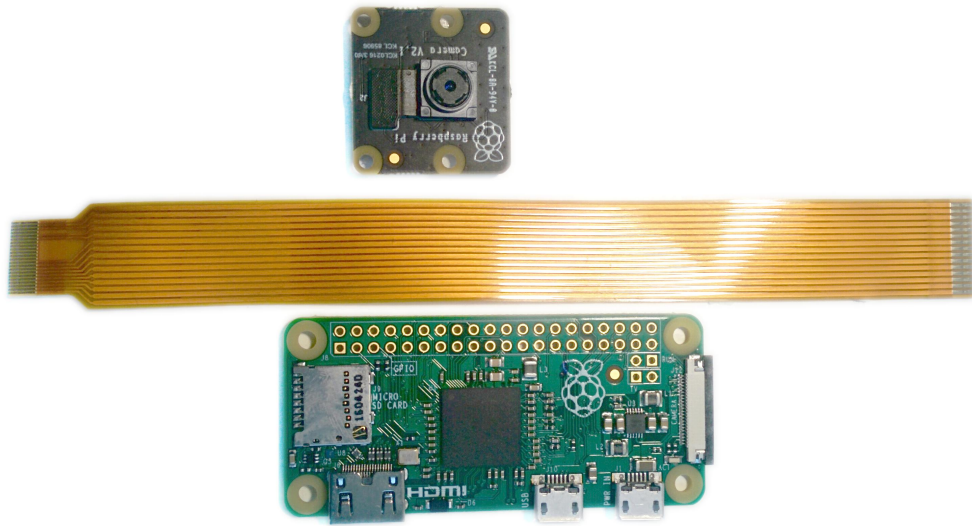
将电缆固定在端口中后，将套环向下按以将电缆锁定到位。如果操作得当，您应该能够轻松地通过相机的电缆提起 Pi 而不会掉落。下图显示了安装良好且方向正确的相机电缆：



请确保相机模块没有放在任何导电的地方（例如树莓派的 USB 端口或其 GPIO 引脚）。

2.1 Pi Zero

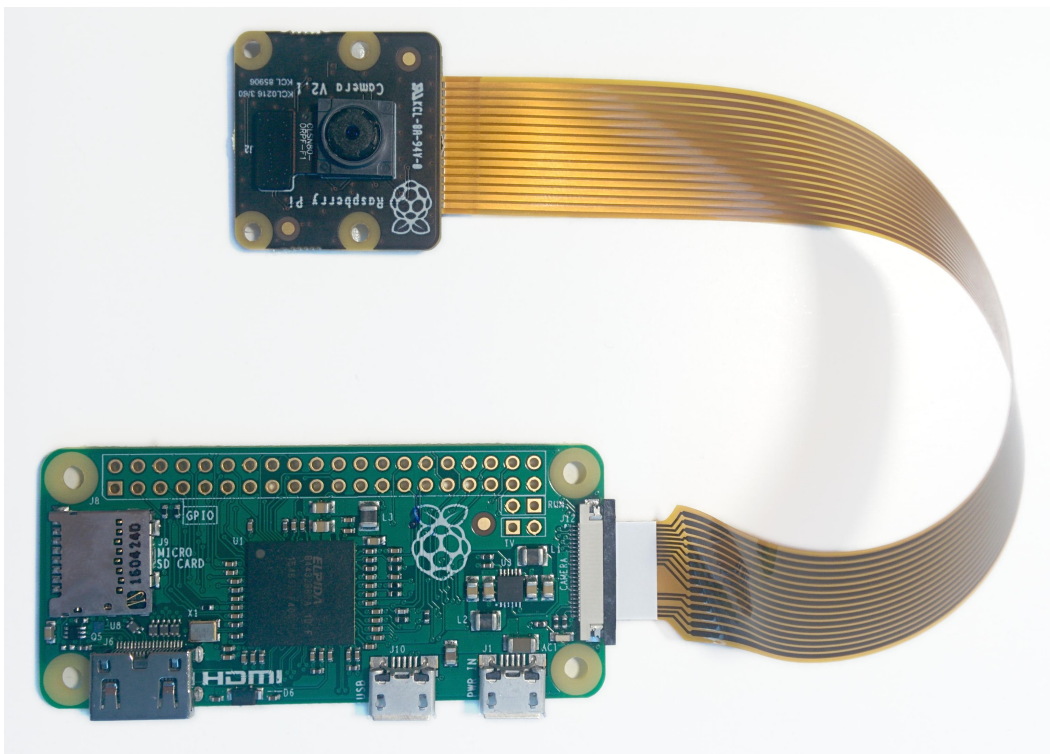
Raspberry Pi Zero 的 1.2 型号包括一个小型 CSI 端口，需要使用相机适配器电缆。



要将相机模块连接到 Pi Zero：

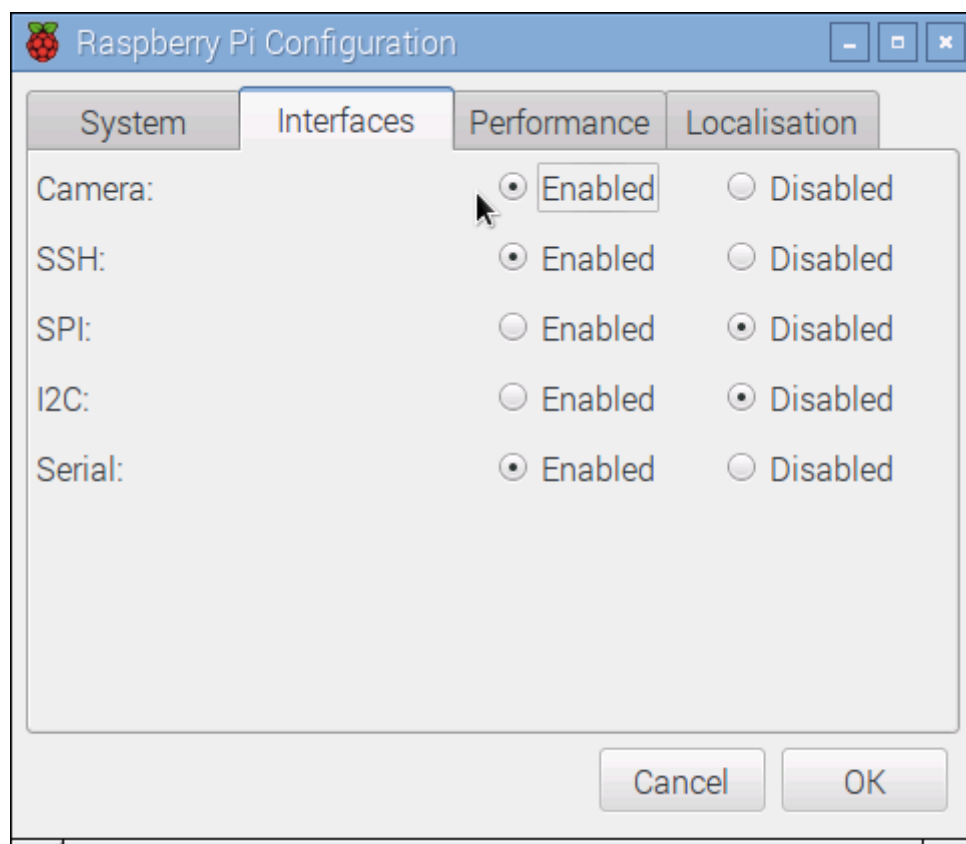
1. 轻轻提起摄像头模块上的卡圈并将电缆拉出，从而卸下现有摄像头模块的电缆。
2. 接下来，插入适配器电缆较宽的一端，导体朝向与相机镜头相同的方向。
3. 最后，通过轻轻提起电路板边缘的套环将适配器连接到 Pi Zero（小心这一点，因为它们比常规 CSI 端口上的项圈更精致）并插入带有导体的适配器的较小端 面对 Pi Zero 的背面。

您的设置应如下所示：



2.2 测试

现在可以将树莓派开机，一旦系统启动完成后，打开树莓派设置页面，勾选相机使能选项。



执行此操作后您将需要重新启动（但这是一次性设置，因此除非您重新安装操作系统或切换 SD 卡，否则您无需再次执行此操作）。重新启动后，启动终端并尝试以下命令：

```
raspistill -o image.jpg
```

如果一切正常，相机应该启动，摄像头采集到的预览图像应该出现在显示器上，该条指令在预览 5 秒后，会在关闭相机之前捕获图像（将其存储为 `image.jpg`）。此后您可以继续进行基本操作的阅读。

如果发生其他情况，请阅读显示的错误消息，并尝试根据消息建议进行调试或调整。如果您的树莓派在您运行此命令后立即重新启动，则代表着您的电源不足以运行树莓派和相机模块（以及您连接的任何其他外围设备）。

三、基本操作

所有掌握 Python 初级使用的程序员都应该可以合理地使用picamera的操作。如果您有其他对于picamera的建议，请不吝指出。

Warning

Python 在检查其他路径之前先检查当前目录，当您尝试导入这些模块时，在现有 Python 模块之后命名脚本会导致错误。因此在尝试本章脚本时，不要将您的文件命名为 `picamera.py`。

3.1 采样图像存储到文件

您可以使用`capture()`函数定义采样图像存储到文件的文件名：

```
from time import sleep
from picamera import PiCamera

camera = PiCamera()
camera.resolution = (1024, 768)
camera.start_preview()
# Camera warm-up time
sleep(2)
camera.capture('foo.jpg')
```

请注意，当使用 `capture()` 方法返回时，由 `picamera` 打开的文件（如上例所示）将会被刷新和关闭，以便其他进程访问该数据。

3.2 采样图像存储到流

您也可以利用将 `capture()` 方法捕获到的图像指定保存为类似文件的对象（`socket()`、`io.BytesIO` 流、现有的打开文件对象等）。

```
from io import BytesIO
from time import sleep
from picamera import PiCamera

# Create an in-memory stream
my_stream = BytesIO()
camera = PiCamera()
camera.start_preview()
# Camera warm-up time
sleep(2)
camera.capture(my_stream, 'jpeg')
```

请注意，在上述情况下明确指定了格式。BytesIO 对象没有文件名，因此相机无法自动确定要使用的格式。

另外要记住的一件事是（与指定文件名不同），捕获后流不会自动关闭；picamera 假设因为它没有打开流，所以也不能假定关闭它。但是，如果对象具有flush方法，则将在捕获返回之前调用此方法。这应该确保一旦捕获返回数据可以被其他进程访问，尽管对象仍然需要关闭：

```
from time import sleep
from picamera import PiCamera

# Explicitly open a new file called my_image.jpg
my_file = open('my_image.jpg', 'wb')
camera = PiCamera()
camera.start_preview()
sleep(2)
camera.capture(my_file)
# At this point my_file.flush() has been called, but the file has
# not yet been closed
my_file.close()
```

请注意，在上述情况下，我们不必指定格式，因为相机会询问 my_file 对象的文件名（具体来说，它会在提供的对象上查找 name 属性）。除了使用 Python 内置的流类（如 BytesIO），您还可以构建自己的自定义输出。

3.3 采样图像存储到PIL图像文件

这是捕获到流的变体。首先，我们将图像捕获到 BytesIO 流（Python 的内存中流类），然后将流的位置倒回到开头，并将流读入 PIL Image 对象：

```
from io import BytesIO
from time import sleep
from picamera import PiCamera
from PIL import Image

# Create the in-memory stream
stream = BytesIO()
camera = PiCamera()
camera.start_preview()
sleep(2)
camera.capture(stream, format='jpeg')
# "Rewind" the stream to the beginning so we can read its content
stream.seek(0)
image = Image.open(stream)
```

3.4 采样图像改变尺寸

有时，特别是在对图像执行某种分析或处理的脚本中，您可能希望捕获比相机当前分辨率更小的图像。尽管可以使用 PIL 或 OpenCV 之类的库来执行这种调整大小，但在捕获图像时让树莓派的 GPU 执行调整大小要高效得多。这项操作可以通过 `capture()` 方法的 `resize` 参数来完成：

```
from time import sleep
from picamera import PiCamera

camera = PiCamera()
camera.resolution = (1024, 768)
camera.start_preview()
# Camera warm-up time
sleep(2)
camera.capture('foo.jpg', resize=(320, 240))
```

使用 `start_recording()` 方法录制视频时也可以指定 `resize` 参数。

3.5 采样图像的一致性

您可能希望拍摄一系列在亮度、颜色和对比度方面看起来都相同的图像（例如，这在延时摄影中很有用）。需要使用各种属性以确保跨多个镜头的一致性。具体来说，你需要确保相机的曝光时间、白平衡和增益都是固定的：

- 要设置合理的曝光时间，需要将 `shutter_speed` 属性设置为合理的值。

- 或者，将iso设置为固定值。
- 要修复曝光增益，让analog_gain和digital_gain设置在合理的值上，然后将exposure_mode设置为'off'。
- 要修复白平衡，请将awb_mode设置为'off'，然后将awb_gains设置为（红色、蓝色）增益元组。

实际使用中，可能很难确定这些属性的合适值。从经验来说，对于iso，白天的合理值在 100 和 200 之间，低光照环境下，iso设置为 400 和 800 更适合。要确定 shutter_speed 的合理值，您可以查询 exposure_speed 的属性。对于曝光增益来说，通常analog_gain大于1时，exposure_mode可以设置为'off'。最后，要确定awb_gains的合理值，只需查询属性，同时将awb_mode设置为'off'以外的其他值。这样，相机的白平衡增益将由自动白平衡算法决定。

以下脚本提供了配置这些设置的简要示例：

```
from time import sleep
from picamera import PiCamera

camera = PiCamera(resolution=(1280, 720), framerate=30)
# Set ISO to the desired value
camera.iso = 100
# Wait for the automatic gain control to settle
sleep(2)
# Now fix the values
camera.shutter_speed = camera.exposure_speed
camera.exposure_mode = 'off'
g = camera.awb_gains
camera.awb_mode = 'off'
camera.awb_gains = g
# Finally, take several photos with the fixed settings
camera.capture_sequence(['image%02d.jpg' % i for i in range(10)])
```

3.6 定时拍摄

进行定时摄影的最简单方法是使用 capture_continuous() 方法。使用这种方法，相机会不断地捕捉图像，直到您告诉它停止为止。图像会自动赋予唯一名称，您可以轻松控制捕获之间的延迟。以下代码显示了如何在每次拍摄之间延迟 5 分钟来捕获图像：

```

from time import sleep
from picamera import PiCamera

camera = PiCamera()
camera.start_preview()
sleep(2)
for filename in camera.capture_continuous('img{counter:03d}.jpg'):
    print('Captured %s' % filename)
    sleep(300) # wait 5 minutes

```

您可能希望在特定时间捕获图像，比如在每小时开始时。这只需要改进循环中的延迟（利用`datetime`模块计算日期和时间；此示例还演示了捕获文件名中的时间戳（timestamp）模板）：

```

from time import sleep
from picamera import PiCamera
from datetime import datetime, timedelta

def wait():
    # Calculate the delay to the start of the next hour
    next_hour = (datetime.now() + timedelta(hour=1)).replace(
        minute=0, second=0, microsecond=0)
    delay = (next_hour - datetime.now()).seconds
    sleep(delay)

camera = PiCamera()
camera.start_preview()
wait()
for filename in camera.capture_continuous('img{timestamp:%Y-%m-%d-%H-%M}.jpg'):
    print('Captured %s' % filename)
    wait()

```

3.7 低光照条件图像采样

树莓派的相机使用图像类技巧，可以在低光照条件下捕捉图像。主要方法是设置高增益和长曝光时间，以使相机收集尽可能多的光线。然而，`shutter_speed`属性受相机帧率的限制，所以需要做的第一件事是设置一个非常慢的帧率`framerate`。以下脚本以 6 秒的曝光时间捕获图像（树莓派的 V1 摄像头模块的最大能够设置 6 秒曝光时间，V2 摄像头模块可以设置 10 秒的曝光时间）：

```

from picamera import PiCamera
from time import sleep

```

```
from fractions import Fraction

# Force sensor mode 3 (the long exposure mode), set
# the framerate to 1/6fps, the shutter speed to 6s,
# and ISO to 800 (for maximum gain)
camera = PiCamera(
    resolution=(1280, 720),
    framerate=Fraction(1, 6),
    sensor_mode=3)
camera.shutter_speed = 6000000
camera.iso = 800
# Give the camera a good long time to set gains and
# measure AWB (you may wish to use fixed AWB instead)
sleep(30)
camera.exposure_mode = 'off'
# Finally, capture an image with a 6s exposure. Due
# to mode switching on the still port, this will take
# longer than 6 seconds
camera.capture('dark.jpg')
```

需要注意的是，在黑暗条件以外的任何情况下，此脚本生成的很可能是全白的或严重过度曝光图像。

Note

树莓派的相机模块使用滚动快门。这意味着如果移动的对象相对于相机移动，它们可能会出现扭曲。使用更长的曝光时间会夸大这种效果。

使用长时间曝光时，最好使用 `framerate_range` 函数而不是 `framerate`。这允许相机动态改变帧速率，并在可能的情况下使用较短的帧速率（导致更短的捕获延迟）。上面的脚本中并未使用这一设置，因为快门速度被强制为 6 秒（V1 相机模块上可能的最大值），使用帧率范围毫无意义。

3.8 网络流中进行图像采样

网络流媒体图像采样是 3.6 定时摄影的一种变体。这里我们有两个脚本：一个服务器（大概在一台快速的机器上），它侦听来自树莓派的连接，以及一个在树莓派上运行并向服务器发送连续图像流的客户端。我们将使用一个非常简单的通信协议：首先将图像的长度作为 32 位整数（以 Little Endian 格式）发送，然后是图像数据的字节。如果长度为 0，则表示连接应该关闭，因为不会有更多图像出现。该协议如下图所示：

Image Length (68702)	Image Data	Image Length (87532)	Image Data	Image Length (0)
4 bytes	68702 bytes	4 bytes	87532 bytes	4 bytes

首先是服务器脚本（它依赖于 PIL 来读取 JPEG，但您可以将其替换为任何其他合适的图形库，例如 OpenCV 或 GraphicsMagick）：

```
import io
import socket
import struct
from PIL import Image

# Start a socket listening for connections on 0.0.0.0:8000 (0.0.0.0
means
# all interfaces)
server_socket = socket.socket()
server_socket.bind(('0.0.0.0', 8000))
server_socket.listen(0)

# Accept a single connection and make a file-like object out of it
connection = server_socket.accept()[0].makefile('rb')
try:
    while True:
        # Read the length of the image as a 32-bit unsigned int. If
the
        # length is zero, quit the loop
        image_len = struct.unpack('<L',
connection.read(struct.calcsize('<L')))[0]
        if not image_len:
            break
        # Construct a stream to hold the image data and read the
image
        # data from the connection
        image_stream = io.BytesIO()
        image_stream.write(connection.read(image_len))
        # Rewind the stream, open it as an image with PIL and do
some
        # processing on it
        image_stream.seek(0)
        image = Image.open(image_stream)
        print('Image is %dx%d' % image.size)
```



```
        image.verify()
        print('Image is verified')
finally:
    connection.close()
    server_socket.close()
```

树莓派上的客户端代码为:

```
import io
import socket
import struct
import time
import picamera

# Connect a client socket to my_server:8000 (change my_server to the
# hostname of your server)
client_socket = socket.socket()
client_socket.connect(('my_server', 8000))

# Make a file-like object out of the connection
connection = client_socket.makefile('wb')
try:
    camera = picamera.PiCamera()
    camera.resolution = (640, 480)
    # Start a preview and let the camera warm up for 2 seconds
    camera.start_preview()
    time.sleep(2)

    # Note the start time and construct a stream to hold image data
    # temporarily (we could write it directly to connection but in
this
    # case we want to find out the size of each capture first to
keep
    # our protocol simple)
    start = time.time()
    stream = io.BytesIO()
    for foo in camera.capture_continuous(stream, 'jpeg'):
        # Write the length of the capture to the stream and flush to
        # ensure it actually gets sent
        connection.write(struct.pack('<L', stream.tell()))
        connection.flush()
        # Rewind the stream and send the image data over the wire
        stream.seek(0)
        connection.write(stream.read())
```

```
# If we've been capturing for more than 30 seconds, quit
if time.time() - start > 30:
    break

# Reset the stream for the next capture
stream.seek(0)
stream.truncate()

# Write a length of zero to the stream to signal we're done
connection.write(struct.pack('<L', 0))
finally:
    connection.close()
    client_socket.close()
```

在本节示例中，应首先运行服务器脚本，确保有一个侦听端口准备接收来自客户端脚本的连接。

3.9 视频保存到文件

保存视频文件代码如下：

```
import picamera

camera = picamera.PiCamera()
camera.resolution = (640, 480)
camera.start_recording('my_video.h264')
camera.wait_recording(60)
camera.stop_recording()
```

请注意，我们在上面的示例中使用了`wait_recording()`而不是我们在上面的图像捕获配方中使用的`time.sleep()`。`wait_recording()`方法的相似之处在于它会暂停指定的秒数，但与`time.sleep()`不同的是，它会在等待时不断检查记录错误（例如磁盘空间不足的情况）。如果我们使用`time.sleep()`代替，则此类错误只会由`stop_recording()`调用引发（可能在错误实际发生之后很久）。

3.10 视频保存到流

这其实与3.9 视频保存到文件非常相像：

```
from io import BytesIO
from picamera import PiCamera

stream = BytesIO()
camera = PiCamera()
camera.resolution = (640, 480)
camera.start_recording(stream, format='h264', quality=23)
camera.wait_recording(15)
camera.stop_recording()
```

这里设置的质量参数是为了让编码器指定保存的图像质量级别。相机的 H.264 编码器主要受两个参数限制：

- 比特率将编码器的输出限制为每秒一定数量的比特。默认为 17000000 (17Mbps)，最大值为 25000000 (25Mbps)。较高的值使编码器有更多的“自由”以更高的质量进行编码。您可能会发现，除非录制更高的分辨率，默认设置不会限制编码器。
- quality告诉编码器要采集什么级别的图像质量。quality的值可以介于 1（最高质量）和 40（最低质量）之间，典型值在 20 和 25 之间，典型值可以提供带宽和质量之间的合理权衡。

除了使用 Python 内置的流类（如 BytesIO），您还可以构建自己的自定义输出。这对于视频录制特别有用，具体示例可以参见4.5 自定义输出。

3.11 视频保存为多个文件

如果您希望将视频保存为多个文件，那么可以采用 split_recording()函数进行视频录制：

```
import picamera

camera = picamera.PiCamera(resolution=(640, 480))
camera.start_recording('1.h264')
camera.wait_recording(5)
for i in range(2, 11):
    camera.split_recording('%d.h264' % i)
    camera.wait_recording(5)
camera.stop_recording()
```

以上代码会生成10个视频文件，分别命名为1.h264、2.h264等等，每段视频长约5秒（split_recording()会自动在关键帧进行分帧）。

record_sequence()函数可以用更简洁的代码来实现视频分段：

```
import picamera

camera = picamera.PiCamera(resolution=(640, 480))
for filename in camera.record_sequence(
    '%d.h264' % i for i in range(1, 11)):
    camera.wait_recording(5)
```

`record_sequence()` 函数从 1.3 版本开始加入函数库中。

3.12 视频保存到循环流中

这类似于将视频录制到流中，但使用 `picamera` 库提供的一种特殊类型的内存流。`PiCameraCircularIO` 类实现了一个基于环形缓冲区的流，专门用于视频录制。这使您能够保留包含最后 `n` 秒录制视频的内存流（其中 `n` 由视频录制的比特率和流下的环形缓冲区的大小决定）。

此类存储的典型用例是安全应用程序，在这些应用程序中，人们希望检测运动并仅将检测到运动的视频记录到磁盘。此示例将 20 秒的视频保留在内存中，直到 `write_now` 函数返回 `True`（在此实现中，这是随机的，但可以例如用某种运动检测算法替换它）。一旦 `write_now` 返回 `True`，脚本将再等待 10 秒（以便缓冲区包含事件前 10 秒和事件后 10 秒的视频）并将结果视频写入磁盘，然后再返回等待：

```
import io
import random
import picamera

def motion_detected():
    # Randomly return True (like a fake motion detection routine)
    return random.randint(0, 10) == 0

camera = picamera.PiCamera()
stream = picamera.PiCameraCircularIO(camera, seconds=20)
camera.start_recording(stream, format='h264')
try:
    while True:
        camera.wait_recording(1)
        if motion_detected():
            # Keep recording for 10 seconds and only then write the
            # stream to disk
            camera.wait_recording(10)
            stream.copy_to('motion.h264')
finally:
    camera.stop_recording()
```

在上面的脚本中，我们使用特殊的`copy_to()`方法将流复制到磁盘文件。这会自动处理诸如在循环缓冲区中查找第一个关键帧的开始等细节，并且还提供诸如写入特定字节数或秒数之类的工具。

Note

请注意，流中至少有 20 秒的视频。如果 H.264 编码器需要低于指定比特率（默认为 17Mbps）来录制视频，则流中将有 20 秒以上的视频可用。

`copy_to()`函数从1.11版本开始加入函数库中。

3.13 视频保存到网络流

这类似于将视频录制到流，但我们将使用从`socket()`创建的类似文件的对象，而不是像`BytesIO`这样的内存流。与捕获到网络流中的示例不同，我们不需要通过编写诸如图像长度之类的内容来使网络协议复杂化。这次我们发送连续的视频帧流（它以一种更有效的方式包含这些信息），所以我们可以简单地将记录直接转储到网络端口。

首先，服务器端脚本将简单地读取视频流并将其传送到媒体播放器进行显示：

```
import socket
import subprocess

# Start a socket listening for connections on 0.0.0.0:8000 (0.0.0.0
# means
# all interfaces)
server_socket = socket.socket()
server_socket.bind(('0.0.0.0', 8000))
server_socket.listen(0)

# Accept a single connection and make a file-like object out of it
connection = server_socket.accept()[0].makefile('rb')
try:
    # Run a viewer with an appropriate command line. Uncomment the
    # mplayer
    # version if you would prefer to use mplayer instead of VLC
    cmdline = ['vlc', '--demux', 'h264', '-']
    #cmdline = ['mplayer', '-fps', '25', '-cache', '1024', '-']
    player = subprocess.Popen(cmdline, stdin=subprocess.PIPE)
    while True:
```

```

        # Repeatedly read 1k of data from the connection and write
it to

        # the media player's stdin
        data = connection.read(1024)
        if not data:
            break
        player.stdin.write(data)
finally:
    connection.close()
    server_socket.close()
    player.terminate()

```

Note

如果您在 Windows 上运行此脚本，您可能需要提供 VLC 或 mplayer 可执行文件的完整路径。如果您在 Mac OS X 上运行此脚本，并使用从 MacPorts 安装的 Python，请确保您还从 MacPorts 安装了 VLC 或 mplayer。

您可能会注意到此设置有几秒钟的延迟。这是正常现象，因为媒体播放器会缓冲几秒钟以防止不可靠的网络流。一些媒体播放器（特别是在这种情况下的 mplayer）允许用户跳到缓冲区的末尾（在 mplayer 中按右光标键），通过增加延迟/丢弃的网络数据包将中断播放的风险来减少延迟。

现在对于客户端脚本，它只是通过从网络套接字创建的类似文件的对象开始记录：

```

import socket
import time
import picamera

# Connect a client socket to my_server:8000 (change my_server to the
# hostname of your server)
client_socket = socket.socket()
client_socket.connect(('my_server', 8000))

# Make a file-like object out of the connection
connection = client_socket.makefile('wb')
try:
    camera = picamera.PiCamera()
    camera.resolution = (640, 480)
    camera.framerate = 24
    # Start a preview and let the camera warm up for 2 seconds
    camera.start_preview()
    time.sleep(2)

```

```
# Start recording, sending the output to the connection for 60
# seconds, then stop
camera.start_recording(connection, format='h264')
camera.wait_recording(60)
camera.stop_recording()
finally:
    connection.close()
    client_socket.close()
```

注意，在 Linux 上，使用netcat和raspivid的组合更容易实现上述效果。例如：

```
# on the server
$ nc -l 8000 | vlc --demux h264 -

# on the client
raspivid -w 640 -h 480 -t 60000 -o - | nc my_server 8000
```

这个方法确实可以作为视频流应用程序的起点。也可以相对容易地扭转这个实例的应用。在这种情况下，树莓派充当服务器，等待来自客户端的连接。当它接受连接时，它会开始通过流的方式传输视频 60 秒。另一个变化（仅用于演示目的）是立即初始化相机而不是等待连接以允许流在连接时更快地启动：

```
import socket
import time
import picamera

camera = picamera.PiCamera()
camera.resolution = (640, 480)
camera.framerate = 24

server_socket = socket.socket()
server_socket.bind(('0.0.0.0', 8000))
server_socket.listen(0)

# Accept a single connection and make a file-like object out of it
connection = server_socket.accept()[0].makefile('wb')
try:
    camera.start_recording(connection, format='h264')
    camera.wait_recording(60)
    camera.stop_recording()
finally:
    connection.close()
```

```
server_socket.close()
```

这种设置的一个优点是客户端不需要脚本——我们可以简单地使用带有网络 URL 的 VLC：

```
vlc tcp/h264://my_pi_address:8000/
```

Note

VLC或 mplayer目前都不能使用 GPU 进行解码，试图在树莓派的 CPU 上执行视频解码时会占用大量CPU资源，树莓派的CPU无法完成这项任务，因此不能在树莓派上播放。您需要在更快的机器上运行这些应用程序（“更快”在这里是一个相对术语：即使是 Atom 驱动上网本也应该足够快以完成非高清分辨率的任务）。

3.14 预览图像的叠加

相机预览系统可以同时操作多个分层渲染器。虽然 picamera 库只允许将单个渲染器连接到相机的预览端口，但它确实允许创建显示静态图像的其他渲染器。这些重叠的渲染器可用于创建简单的用户界面。

Note

重叠图像不会出现在图像捕获或视频录制中。如果您需要在相机的输出中嵌入附加信息，请参阅3.15 在输出上叠加文本。

使用叠加渲染器的一个难点是它们期望填充到相机块大小的未编码 RGB 输入。相机的块大小为 32x16，因此提供给渲染器的任何图像数据的宽度必须是 32 的倍数，高度必须是 16 的倍数。预期的特定 RGB 格式是交错的无符号字节。这一切听起来很复杂，在实践中非常简单。

以下示例演示了使用 PIL 加载任意大小的图像，将其填充到所需的大小，并为调用add_overlay()生成未编码的 RGB 数据：

```
import picamera
from PIL import Image
from time import sleep

camera = picamera.PiCamera()
camera.resolution = (1280, 720)
camera.framerate = 24
camera.start_preview()

# Load the arbitrarily sized image
img = Image.open('overlay.png')

# Create an image padded to the required size with
```



```

# mode 'RGB'
pad = Image.new('RGB', (
    ((img.size[0] + 31) // 32) * 32,
    ((img.size[1] + 15) // 16) * 16,
))
# Paste the original image into the padded one
pad.paste(img, (0, 0))

# Add the overlay with the padded image as the source,
# but the original image's dimensions
o = camera.add_overlay(pad.tobytes(), size=img.size)
# By default, the overlay is in layer 0, beneath the
# preview (which defaults to layer 2). Here we make
# the new overlay semi-transparent, then move it above
# the preview
o.alpha = 128
o.layer = 3

# Wait indefinitely until the user terminates the script
while True:
    sleep(1)

```

或者，您可以直接从 numpy 数组生成叠加层，而不是使用图像文件作为源。在下面的例子中，我们构建了一个与屏幕分辨率相同的 numpy 数组，然后通过中心绘制一个白色的十字并将其作为一个简单的十字线覆盖在预览上：

```

import time
import picamera
import numpy as np

# Create an array representing a 1280x720 image of
# a cross through the center of the display. The shape of
# the array must be of the form (height, width, color)
a = np.zeros((720, 1280, 3), dtype=np.uint8)
a[360, :, :] = 0xff
a[:, 640, :] = 0xff

camera = picamera.PiCamera()
camera.resolution = (1280, 720)
camera.framerate = 24
camera.start_preview()

# Add the overlay directly into layer 3 with transparency;
# we can omit the size parameter of add_overlay as the

```

```
# size is the same as the camera's resolution
o = camera.add_overlay(np.getbuffer(a), layer=3, alpha=64)
try:
    # Wait indefinitely until the user terminates the script
    while True:
        time.sleep(1)
finally:
    camera.remove_overlay(o)
```

构建简单的用户界面可以考虑：通过将可以隐藏覆盖的渲染器移动到默认为 2 的预览层下方、使用alpha属性使其半透明，并调整大小使其不填满屏幕。

本函数自1.8版本后加入函数库中。

3.15 在输出图像上加入文字

相机包括一个基本的注释工具，允许在所有输出（包括预览、图像捕获和视频录制）上覆盖最多 255 个字符的 ASCII 文本。为此，只需为 `annotate_text` 属性分配一个字符串：

```
import picamera
import time

camera = picamera.PiCamera()
camera.resolution = (640, 480)
camera.framerate = 24
camera.start_preview()
camera.annotate_text = 'Hello world!'
time.sleep(2)
# Take a picture including the annotation
camera.capture('foo.jpg')
```

稍微使用一些技巧，就可以显示更长的字符串：

```
import picamera
import time
import itertools

s = "This message would be far too long to display normally..."

camera = picamera.PiCamera()
camera.resolution = (640, 480)
camera.framerate = 24
```

```
camera.start_preview()
camera.annotate_text = ' ' * 31
for c in itertools.cycle(s):
    camera.annotate_text = camera.annotate_text[1:31] + c
    time.sleep(0.1)
```

还可以在录像视频中显示（和嵌入）时间戳（本节还演示了在时间戳后面绘制背景以与`annotate_background`属性进行对比）：

```
import picamera
import datetime as dt

camera = picamera.PiCamera(resolution=(1280, 720), framerate=24)
camera.start_preview()
camera.annotate_background = picamera.Color('black')
camera.annotate_text = dt.datetime.now().strftime('%Y-%m-%d
%H:%M:%S')
camera.start_recording('timestamped.h264')
start = dt.datetime.now()
while (dt.datetime.now() - start).seconds < 30:
    camera.annotate_text = dt.datetime.now().strftime('%Y-%m-%d
%H:%M:%S')
    camera.wait_recording(0.2)
camera.stop_recording()
```

本功能自1.7版本后加入函数库。

3.16 控制LED

在某些情况下，您可能会发现 V1 摄像头模块的红色 LED 是一个影响拍摄的器件（V2 摄像头模块没有 LED）。例如，在自动特写野生动物摄影的情况下，LED 可能会吓跑动物。它还可能对特写对象造成不必要的反射红色眩光。

解决这个问题的一种简单方法是在 LED 上放置一些不透明的覆盖物（例如蓝色胶粘剂或电工胶带）。另一种方法是在引导配置中使用 `disable_camera_led` 选项。

但是，如果您安装了 RPi.GPIO 包，并且您的 Python 进程以足够的权限运行（通常这意味着以 root 身份运行 `sudo python`），您还可以通过 `led` 属性控制 LED：

```
import picamera

camera = picamera.PiCamera()
# Turn the camera's LED off
camera.led = False
# Take a picture while the LED remains off
camera.capture('foo.jpg')
```

Note

当摄像头模块连接到树莓派 3 B 型时，该函数无法控制摄像头 LED，因为控制 LED 的 GPIO 已移至 ARM 处理器，无法直接访问的 GPIO 扩展器。

Warning

请注意，当您第一次使用 LED 属性时，它将使用 `GPIO.setmode(GPIO.BCM)` 将 GPIO 库设置为 Broadcom (BCM) 模式，并使用 `GPIO.setwarnings(False)` 禁用警告。在板载模式下，无法控制 LED。

四、进阶操作

本章内容可能不适合初学者，请随时提出改进意见或其他操作建议。

Warning

Python 在检查其他路径之前先检查当前目录，当您尝试导入这些模块时，在现有 Python 模块之后命名脚本会导致错误。因此在尝试本章脚本时，不要将您的文件命名为 `picamera.py`。

4.1 采样图像到numpy array

从 1.11 版本开始，picamera 可以直接捕获到任何支持 Python 缓冲协议的对象（包括 numpy 的 `ndarray`）。只需将对象作为捕获的目的地传递，图像数据将直接写入对象。目标对象必须满足各种要求（其中一些取决于您使用的 Python 版本）：

1. 缓冲区对象必须是可写的（例如，您不能捕获到字节对象，因为它是不可变的）。
2. 缓冲区对象必须足够大以接收所有图像数据。
3. 缓冲区对象必须是一维的。（仅限 Python 2.x）
4. 缓冲区对象必须具有字节大小的项目。（仅限 Python 2.x）

例如，要直接捕获到三维 numpy `ndarray`（仅限 Python 3.x）：

```
import time
import picamera
import numpy as np

with picamera.PiCamera() as camera:
    camera.resolution = (320, 240)
    camera.framerate = 24
    time.sleep(2)
    output = np.empty((240, 320, 3), dtype=np.uint8)
    camera.capture(output, 'rgb')
```

同样重要的是要注意，当输出为未编码格式时，相机会舍入请求的分辨率。水平分辨率向上舍入到最接近的 32 像素的倍数，而垂直分辨率向上舍入到最接近的 16 像素的倍数。例如，如果请求的分辨率为 100x100，则捕获实际上将包含 128x112 像素的数据，但超过 100x100 的像素将不会被初始化。

因此，要捕获 100x100 的图像，我们首先需要提供一个 128x112 的数组，然后剥离未初始化的像素。以下示例演示了这一点以及在 Python 2.x 下所需的代码：

```
import time
import picamera
import numpy as np

with picamera.PiCamera() as camera:
    camera.resolution = (100, 100)
    camera.framerate = 24
    time.sleep(2)
    output = np.empty((112 * 128 * 3,), dtype=np.uint8)
    camera.capture(output, 'rgb')
    output = output.reshape((112, 128, 3))
    output = output[:100, :100, :]
```

Warning

在某些情况下（未调整大小、非 YUV、视频端口捕获），分辨率四舍五入为 16x16 块而不是 32x16。需要相应地调整相机的分辨率。

自1.11版本后加入函数库。

4.2 使用OpenCV对象进行图像采集

这是捕获到 numpy 数组的变体。OpenCV 使用 numpy 数组作为图像并默认为平面 BGR 中的颜色。因此，以下是捕获 OpenCV 兼容图像所需的全部内容：

```
import time
import picamera
import numpy as np
import cv2

with picamera.PiCamera() as camera:
    camera.resolution = (320, 240)
    camera.framerate = 24
    time.sleep(2)
    image = np.empty((240 * 320 * 3,), dtype=np.uint8)
    camera.capture(image, 'bgr')
    image = image.reshape((240, 320, 3))
```

1.11版本后重置了直接数组。

4.3 未编码的图像 (YUV格式)

如果您希望捕获图像不丢失细节（由于 JPEG 的有损压缩），您可能最好探索 PNG 作为替代图像格式（PNG 使用无损压缩）。但是，某些应用程序（尤其是科学应用程序）只需要数字形式的图像数据。为此，提供了“YUV”格式：

```
import time
import picamera

with picamera.PiCamera() as camera:
    camera.resolution = (100, 100)
    camera.start_preview()
    time.sleep(2)
    camera.capture('image.data', 'yuv')
```

使用的特定 YUV 格式是 YUV420（平面）。这意味着 Y（亮度）值首先出现在结果数据中并具有全分辨率（图像中的每个像素有一个 1 字节的 Y 值）。Y 值之后是 U（色度）值，最后是 V（色度）值。UV 值具有 Y 分量分辨率的四分之一（每个 1 字节 U 值和 1 字节 V 值的正方形中的 4 个 1 字节 Y 值）。如下图所示：

同样重要的是要注意，当输出为未编码格式时，相机会舍入请求的分辨率。水平分辨率向上舍入到最接近的 32 像素的倍数，而垂直分辨率向上舍入到最接近的 16 像素的倍数。例如，如果请求的分辨率为 100x100，则捕获实际上将包含 128x112 像素的数据，但超过 100x100 的像素将不会被初始化。

假设 YUV420 格式包含每个像素 1.5 字节的数据（每个像素 1 字节 Y 值，每 4 个像素 1 字节 U 和 V 值），并考虑到分辨率舍入，100x100 YUV 捕获将是：

128.0	100 rounded up to nearest multiple of 32
× 112.0	100 rounded up to nearest multiple of 16
× 1.5	bytes of data per pixel in YUV420 format
21504.0	bytes total

数据的前 14336 字节 (128*112) 将是 Y 值，接下来的 3584 字节 (128 \times 112 \div 4) 将是 U 值，最后 3584 字节将是 V 值。

以下代码演示了捕获 YUV 图像数据，将数据加载到一组 numpy 数组中，并以有效的方式将数据转换为 RGB 格式：

```
from __future__ import division
```

```

import time
import picamera
import numpy as np

width = 100
height = 100
stream = open('image.data', 'w+b')
# Capture the image in YUV format
with picamera.PiCamera() as camera:
    camera.resolution = (width, height)
    camera.start_preview()
    time.sleep(2)
    camera.capture(stream, 'yuv')
# Rewind the stream for reading
stream.seek(0)
# Calculate the actual image size in the stream (accounting for
rounding
# of the resolution)
fwidth = (width + 31) // 32 * 32
fheight = (height + 15) // 16 * 16
# Load the Y (luminance) data from the stream
Y = np.fromfile(stream, dtype=np.uint8, count=fwidth*fheight).\
    reshape((fheight, fwidth))
# Load the UV (chrominance) data from the stream, and double its
size
U = np.fromfile(stream, dtype=np.uint8, count=(fwidth//2)*
(fheight//2)).\
    reshape((fheight//2, fwidth//2)).\
    repeat(2, axis=0).repeat(2, axis=1)
V = np.fromfile(stream, dtype=np.uint8, count=(fwidth//2)*
(fheight//2)).\
    reshape((fheight//2, fwidth//2)).\
    repeat(2, axis=0).repeat(2, axis=1)
# Stack the YUV channels together, crop the actual resolution,
convert to
# floating point for later calculations, and apply the standard
biases
YUV = np.dstack((Y, U, V))[:height, :width, :].astype(np.float)
YUV[:, :, 0] = YUV[:, :, 0] - 16 # Offset Y by 16
YUV[:, :, 1:] = YUV[:, :, 1:] - 128 # Offset UV by 128
# YUV conversion matrix from ITU-R BT.601 version (SDTV)
#
        Y        U        V

```



```
M = np.array([[1.164, 0.000, 1.596],    # R
              [1.164, -0.392, -0.813],  # G
              [1.164, 2.017, 0.000]])    # B

# Take the dot product with the matrix to produce RGB output, clamp
the
# results to byte range and convert to bytes
RGB = YUV.dot(M.T).clip(0, 255).astype(np.uint8)
```

Note

您可能会注意到我们在上面的代码中使用了 `open()` 而不是其他示例中的 `io.open()`。这是因为 `numpy` 的 `numpy.fromfile()` 方法只接受“真实”的文件对象。

这个示例现在被封装在 `picamera.array` 模块的 `PiYUVArray` 类中，这意味着同样可以实现如下：

```
import time
import picamera
import picamera.array

with picamera.PiCamera() as camera:
    with picamera.array.PiYUVArray(camera) as stream:
        camera.resolution = (100, 100)
        camera.start_preview()
        time.sleep(2)
        camera.capture(stream, 'yuv')
        # Show size of YUV data
        print(stream.array.shape)
        # Show size of RGB converted data
        print(stream.rgb_array.shape)
```

从 1.11 版本开始，您还可以直接捕获到 `numpy` 数组（请参阅 4.1 采样图像到 `numpy` 数组）。由于 Y 和 UV 部分的分辨率不同，这不是直接有用的（如果您需要所有三个组件，最好使用 `PiYUVArray`，因为为了方便起见，它会重新缩放 UV 组件）。但是，如果您只需要 Y 平面，则可以为该平面提供一个足够大的缓冲区，并忽略写入缓冲区时发生的错误（`picamera` 会在引发异常以支持之前故意写入尽可能多的缓冲区）这个用例）：

```
import time
import picamera
import picamera.array
import numpy as np
```

```

with picamera.PiCamera() as camera:
    camera.resolution = (100, 100)
    time.sleep(2)
    y_data = np.empty((112, 128), dtype=np.uint8)
    try:
        camera.capture(y_data, 'yuv')
    except IOError:
        pass
    y_data = y_data[:100, :100]
    # y_data now contains the Y-plane only

```

或者参阅4.4未编码图像捕获（RGB 格式）以了解让相机直接输出RGB数据的方法。

Note

捕获所谓的“原始”格式（'yuv'、'rgb'等）不会提供来自相机传感器的原始拜耳数据。相反，它提供对GPU处理之后、格式编码（JPEG、PNG等）之前的图像数据的访问。目前，访问原始bayer数据的唯一方法是通过capture()方法的bayer参数。有关详细信息，请参阅4.16原始拜耳数据获取。

1.0 版更新：raw_format属性现在已弃用，capture()方法的"raw"格式规范也是如此。只需使用'yuv'格式，如上面的代码所示。

1.5 版更改：添加了关于新picamera.array模块的注释。

1.11 版更改：添加了直接阵列捕获的说明。

4.4 未编码的图像（RGB格式）

RGB 格式比上一节中讨论的 YUV 格式大得多，但对大多数分析更有用。要让相机以 RGB 格式生成输出，您只需将capture()方法的格式指定为'rgb'：

```

import time
import picamera

with picamera.PiCamera() as camera:
    camera.resolution = (100, 100)
    camera.start_preview()
    time.sleep(2)
    camera.capture('image.data', 'rgb')

```

RGB 数据的大小可以与 YUV 捕获类似地计算。首先适当地舍入分辨率（有关详细信息，请参阅4.3未编码的图像（YUV 格式）），然后将像素数乘以 3（1 个字节的红色，1 个字节的绿色和 1 个字节的蓝色强度）。因此，对于 100x100 的捕获，产生的数据量是：

$$\begin{array}{rcl} 128.0 & 100 \text{ rounded up to nearest multiple of } 32 & \\ \times 112.0 & 100 \text{ rounded up to nearest multiple of } 16 & \\ \times 3.0 & \text{bytes of data per pixel in RGB format} & \\ \hline 43008.0 & \text{bytes total} & \end{array}$$

Warning

在某些情况下（未调整大小、非 YUV、视频端口捕获），分辨率四舍五入为 16x16 块而不是 32x16。

产生的 RGB 数据是交错的。也就是说，给定像素的红色、绿色和蓝色值按该顺序组合在一起。数据的第一个字节是 (0, 0) 处像素的红色值，第二个字节是同一像素的绿色值，第三个字节是该像素的蓝色值。第四个字节是 (1, 0) 处像素的红色值，依此类推。

由于 RGB 数据中的平面大小相同（与 YUV420 相比），因此直接捕获到 numpy 数组中是微不足道的（仅限 Python 3.x；请参阅4.1采集到numpy数组的Python 2.x 指令）：

```
import time
import picamera
import picamera.array
import numpy as np

with picamera.PiCamera() as camera:
    camera.resolution = (100, 100)
    time.sleep(2)
    image = np.empty((128, 112, 3), dtype=np.uint8)
    camera.capture(image, 'rgb')
    image = image[:100, :100]
```

Note

从静态端口捕获的 RGB 无法在相机的全分辨率下工作（它们会导致内存不足错误）。如果需要全分辨率，请使用 YUV 捕获或从视频端口捕获。

1.0 版更新：raw_format 属性现在已弃用，capture() 方法的 "raw" 格式规范也是如此。只需使用 'rgb' 格式，如上面的代码所示。

1.5 版更改：添加了关于新picamera.array模块的注释。

1.11 版更改: 添加了直接阵列捕获的说明。

4.5 自定义输出

`picamera` 库中接受文件名的所有方法也接受类似文件的对象。通常，这仅用于实际文件对象或内存流（如 `io.BytesIO`）。但是，构建自定义输出对象非常简单，并且在某些情况下非常有用。类文件对象（就 `picamera` 而言）只是一个具有 `write` 方法的对象，该方法必须接受由字节字符串组成的单个参数，并且可以选择返回写入的字节数。该对象可以选择实现一个 `flush` 方法（它没有参数），它将在输出结束时调用。

自定义输出对于视频录制特别有用，因为自定义输出的 `write` 方法将针对输出的每一帧调用（至少）一次，允许您实现对每一帧做出反应的代码，而无需费心去完全自定义编码器。但是，应该记住，因为 `write` 方法被频繁调用，所以它的实现必须足够快，不会使编码器停顿（如果您愿意，它必须在下一次写入到达之前执行其处理并返回）以避免丢帧）。

以下简单示例演示了一个非常简单的自定义输出，它在计算本应写入的字节数并在输出末尾打印时简单地丢弃输出：

```
import picamera

class MyOutput(object):
    def __init__(self):
        self.size = 0

    def write(self, s):
        self.size += len(s)

    def flush(self):
        print('%d bytes would have been written' % self.size)

with picamera.PiCamera() as camera:
    camera.resolution = (640, 480)
    camera.framerate = 60
    camera.start_recording(MyOutput(), format='h264')
    camera.wait_recording(10)
    camera.stop_recording()
```

以下示例展示了如何使用自定义输出来构建粗略的运动检测系统。我们构造了一个自定义输出对象，用作运动矢量数据的目的地（这特别简单，因为运动矢量数据总是作为单个块到达；相比之下，帧数据有时以几个单独的块到达）。输出对象实际上并没有在任何地方写入运动数据；相反，它将它加载到一个 `numpy` 数组中，并分析数据中是否有任何非常大的

向量，如果有，则向控制台打印一条消息。由于我们不关心在这个例子中保留实际的视频输出，我们使用/dev/null作为视频数据的目的地：

```
from __future__ import division

import picamera
import numpy as np

motion_dtype = np.dtype([
    ('x', 'i1'),
    ('y', 'i1'),
    ('sad', 'u2'),
])

class MyMotionDetector(object):
    def __init__(self, camera):
        width, height = camera.resolution
        self.cols = (width + 15) // 16
        self.cols += 1 # there's always an extra column
        self.rows = (height + 15) // 16

    def write(self, s):
        # Load the motion data from the string to a numpy array
        data = np.fromstring(s, dtype=motion_dtype)
        # Re-shape it and calculate the magnitude of each vector
        data = data.reshape((self.rows, self.cols))
        data = np.sqrt(
            np.square(data['x'].astype(np.float)) +
            np.square(data['y'].astype(np.float))
        ).clip(0, 255).astype(np.uint8)
        # If there're more than 10 vectors with a magnitude greater
        # than 60, then say we've detected motion
        if (data > 60).sum() > 10:
            print('Motion detected!')
        # Pretend we wrote all the bytes of s
        return len(s)

with picamera.PiCamera() as camera:
    camera.resolution = (640, 480)
    camera.framerate = 30
    camera.start_recording(
        # Throw away the video data, but make sure we're using H.264
        '/dev/null', format='h264',
```

```

        # Record motion data to our custom output object
        motion_output=MyMotionDetector(camera)
    )
camera.wait_recording(30)
camera.stop_recording()

```

您可能希望研究 `picamera.array` 模块中的类，这些类实现了几个自定义输出，用于使用 `numpy` 分析数据。 `PiMotionAnalysis` 类可用于从上述示例中删除大部分代码：

```

import picamera
import picamera.array
import numpy as np

class MyMotionDetector(picamera.array.PiMotionAnalysis):
    def analyse(self, a):
        a = np.sqrt(
            np.square(a['x'].astype(np.float)) +
            np.square(a['y'].astype(np.float))
        ).clip(0, 255).astype(np.uint8)

        # If there're more than 10 vectors with a magnitude greater
        # than 60, then say we've detected motion
        if (a > 60).sum() > 10:
            print('Motion detected!')

with picamera.PiCamera() as camera:
    camera.resolution = (640, 480)
    camera.framerate = 30
    camera.start_recording(
        '/dev/null', format='h264',
        motion_output=MyMotionDetector(camera)
    )
    camera.wait_recording(30)
    camera.stop_recording()

```

1.5版本加入的新功能。

4.6 非常规文件输出

如前所示，`picamera`接受多种输出格式。

- 一个字符串，将被视为文件名。
- 一个类似文件的对象，例如：由 `open()` 返回。

- 自定义输出。
- 任何实现缓冲区接口的可变对象。

其中最简单的文件名隐藏了一定的复杂性。准确了解 picamera 如何处理文件非常重要，尤其是在处理“非常规”文件（例如 pipes、FIFO 等）时。

当给定文件名时，picamera 执行以下操作：

1. 以 'wb' 模式打开指定的文件，即以二进制模式打开写入，首先截断文件。
2. 文件以大于正常的缓冲区大小打开，特别是 64Kb。使用大缓冲区大小是因为它在大多数用例中提高了性能和系统负载，即将视频顺序写入磁盘。
3. 请求的数据（图像捕获、视频录制等）将写入打开的文件。
4. 最后，文件被刷新并关闭。请注意，因为 picamera 为您打开了输出，这是 picamera 假定为您关闭输出的唯一情况。

如上所述，这非常适合大多数用例（按顺序将视频写入文件）。但是，如果您通过 FIFO 将数据通过管道传输到另一个进程（picamera 会将其视为任何其他文件），您可能希望避免所有缓冲。在这种情况下，您可以简单地自己打开输出而无需缓冲。如上所述，当您完成输出时，您将负责关闭它（您打开它，因此关闭它的责任也是您的）。

例如：

```
import io
import os
import picamera

with picamera.PiCamera(resolution='VGA') as camera:
    os.mkfifo('video_fifo')
    f = io.open('video_fifo', 'wb', buffering=0)
    try:
        camera.start_recording(f, format='h264')
        camera.wait_recording(10)
        camera.stop_recording()
    finally:
        f.close()
        os.unlink('video_fifo')
```

4.7 快速采样与处理

通过利用其带有 JPEG 编码器的视频捕获功能（通过`use_video_port`参数），该相机能够非常快速地捕获一系列图像。但是，使用此技术有几点需要注意：

- 使用基于视频端口的捕获时，仅捕获视频记录区域；在某些情况下，这可能小于正常的图像捕获区域（参见6.2传感器模式中的讨论）。
- 通过视频端口捕获的 JPEG 图像中没有嵌入 Exif 信息。
- 使用这种技术，采样图像通常会显得具有“颗粒状”。可能需要从静止端口捕获使用更慢、更密集的降噪算法。

所有捕获方法都支持`use_video_port`选项，但这些方法在快速捕获连续帧的能力方面有所不同。因此，虽然`capture()`和`capture_continuous()`都支持`use_video_port`，但`capture_sequence()`是迄今为止最快的方法（因为它不会在每次捕获之前重新初始化编码器）。使用这种方法，作者管理了 1024x768 分辨率的 30fps JPEG 捕获。

默认情况下，`capture_sequence()`特别适合快速捕获固定数量的帧，如以下示例中捕获 5 张图像的代码：

```
import time
import picamera

with picamera.PiCamera() as camera:
    camera.resolution = (1024, 768)
    camera.framerate = 30
    camera.start_preview()
    time.sleep(2)
    camera.capture_sequence([
        'image1.jpg',
        'image2.jpg',
        'image3.jpg',
        'image4.jpg',
        'image5.jpg',
    ], use_video_port=True)
```

我们可以通过使用生成器表达式来提供用于处理的文件名而不是手动指定每个文件名来稍微改进它：

```
import time
import picamera

frames = 60
```



```

with picamera.PiCamera() as camera:
    camera.resolution = (1024, 768)
    camera.framerate = 30
    camera.start_preview()
    # Give the camera some warm-up time
    time.sleep(2)
    start = time.time()
    camera.capture_sequence([
        'image%02d.jpg' % i
        for i in range(frames)
    ], use_video_port=True)
    finish = time.time()
print('Captured %d frames at %.2ffps' % (
    frames,
    frames / (finish - start)))

```

然而，在满足某些条件之前，这仍然不能让我们捕获任意数量的帧。为此，我们需要使用生成器函数向`capture_sequence()`方法提供文件名列表（或更有用的是“流”）：

```

import time
import picamera

frames = 60

def filenames():
    frame = 0
    while frame < frames:
        yield 'image%02d.jpg' % frame
        frame += 1

with picamera.PiCamera(resolution='720p', framerate=30) as camera:
    camera.start_preview()
    # Give the camera some warm-up time
    time.sleep(2)
    start = time.time()
    camera.capture_sequence(filenames(), use_video_port=True)
    finish = time.time()
print('Captured %d frames at %.2ffps' % (
    frames,
    frames / (finish - start)))

```

快速捕获的主要问题首先是 Raspberry Pi 的 IO 带宽极其有限，其次，作为一种格式，JPEG 的效率远低于 H.264 视频格式（也就是说，对于相同数量的字节，H.264 将在相同数量的帧上提供更好的质量）。在更高的分辨率下（超过 800x600），您可能会发现无法在很长时间内（在耗尽磁盘缓存之前）以 30fps 的速度捕获到 Pi 的 SD 卡。

如果您打算在捕获后对帧进行处理，您最好只从生成的文件中捕获视频和解码帧，而不是处理单个 JPEG 捕获。幸运的是，这相对容易，因为 JPEG 格式有一个简单的魔术数 (FF D8)。这意味着我们可以使用自定义输出通过检查每个缓冲区的前两个字节来将帧从 MJPEG 视频记录中分离出来：

```
import io
import time
import picamera

class SplitFrames(object):
    def __init__(self):
        self.frame_num = 0
        self.output = None

    def write(self, buf):
        if buf.startswith(b'\xff\xd8'):
            # Start of new frame; close the old one (if any) and
            # open a new output
            if self.output:
                self.output.close()
            self.frame_num += 1
            self.output = io.open('image%02d.jpg' % self.frame_num,
                                  'wb')
            self.output.write(buf)

with picamera.PiCamera(resolution='720p', framerate=30) as camera:
    camera.start_preview()
    # Give the camera some warm-up time
    time.sleep(2)
    output = SplitFrames()
    start = time.time()
    camera.start_recording(output, format='mjpeg')
    camera.wait_recording(2)
    camera.stop_recording()
    finish = time.time()
    print('Captured %d frames at %.2ffps' % (
        output.frame_num,
```

```
output.frame_num / (finish - start)))
```

到目前为止，我们刚刚将捕获的帧保存到磁盘。稍后使用另一个脚本进行数据处理，但是如果我们想在当前脚本中执行所有处理怎么办？在这种情况下，我们可能根本不需要涉及磁盘（或网络）。我们可以设置一个并行线程池来接受和处理捕获的图像流：

```
import io
import time
import threading
import picamera

class ImageProcessor(threading.Thread):
    def __init__(self, owner):
        super(ImageProcessor, self).__init__()
        self.stream = io.BytesIO()
        self.event = threading.Event()
        self.terminated = False
        self.owner = owner
        self.start()

    def run(self):
        # This method runs in a separate thread
        while not self.terminated:
            # Wait for an image to be written to the stream
            if self.event.wait(1):
                try:
                    self.stream.seek(0)
                    # Read the image and do some processing on it
                    #Image.open(self.stream)
                    #...
                    #...
                    # Set done to True if you want the script to
terminate
                    # at some point
                    #self.owner.done=True
                finally:
                    # Reset the stream and event
                    self.stream.seek(0)
                    self.stream.truncate()
                    self.event.clear()
                    # Return ourselves to the available pool
                    with self.owner.lock:
                        self.owner.pool.append(self)
```

```

class ProcessOutput(object):
    def __init__(self):
        self.done = False

        # Construct a pool of 4 image processors along with a lock
        # to control access between threads
        self.lock = threading.Lock()
        self.pool = [ImageProcessor(self) for i in range(4)]
        self.processor = None

    def write(self, buf):
        if buf.startswith(b'\xff\xd8'):
            # New frame; set the current processor going and grab
            # a spare one
            if self.processor:
                self.processor.event.set()
            with self.lock:
                if self.pool:
                    self.processor = self.pool.pop()
                else:
                    # No processor's available, we'll have to skip
                    # this frame; you may want to print a warning
                    # here to see whether you hit this case
                    self.processor = None
            if self.processor:
                self.processor.stream.write(buf)

    def flush(self):
        # When told to flush (this indicates end of recording), shut
        # down in an orderly fashion. First, add the current
processor
        # back to the pool
        if self.processor:
            with self.lock:
                self.pool.append(self.processor)
                self.processor = None
        # Now, empty the pool, joining each thread as we go
        while True:
            with self.lock:
                try:
                    proc = self.pool.pop()
                except IndexError:
                    pass # pool is empty

```

```

        proc.terminated = True
        proc.join()

with picamera.PiCamera(resolution='VGA') as camera:
    camera.start_preview()
    time.sleep(2)
    output = ProcessOutput()
    camera.start_recording(output, format='mjpeg')
    while not output.done:
        camera.wait_recording(1)
    camera.stop_recording()

```

4.8 采集未编码视频

正如未编码的 RGB 数据可以捕获为图像一样，树莓派的摄像头模块也可以捕获未编码的 RGB（或 YUV）视频数据流。将此与自定义输出中提供的方法（通过 `picamera.array` 中的类）相结合，我们可以生成一个相当快速的颜色检测脚本：

```

import picamera
import numpy as np
from picamera.array import PiRGBAnalysis
from picamera.color import Color

class MyColorAnalyzer(PiRGBAnalysis):
    def __init__(self, camera):
        super(MyColorAnalyzer, self).__init__(camera)
        self.last_color = ''

    def analyze(self, a):
        # Convert the average color of the pixels in the middle box
        c = Color(
            r=int(np.mean(a[30:60, 60:120, 0])),
            g=int(np.mean(a[30:60, 60:120, 1])),
            b=int(np.mean(a[30:60, 60:120, 2]))
        )

        # Convert the color to hue, saturation, lightness
        h, l, s = c.hls
        c = 'none'
        if s > 1/3:
            if h > 8/9 or h < 1/36:
                c = 'red'
            elif 5/9 < h < 2/3:

```

```

        c = 'blue'
    elif 5/36 < h < 4/9:
        c = 'green'

    # If the color has changed, update the display
    if c != self.last_color:
        self.camera.annotate_text = c
        self.last_color = c

with picamera.PiCamera(resolution='160x90', framerate=24) as camera:
    # Fix the camera's white-balance gains
    camera.awb_mode = 'off'
    camera.awb_gains = (1.4, 1.5)

    # Draw a box over the area we're going to watch
    camera.start_preview(alpha=128)
    box = np.zeros((96, 160, 3), dtype=np.uint8)
    box[30:60, 60:120, :] = 0x80
    camera.add_overlay(memoryview(box), size=(160, 90), layer=3,
alpha=64)

    # Construct the analysis output and start recording data to it
    with MyColorAnalyzer(camera) as analyzer:
        camera.start_recording(analyzer, 'rgb')
        try:
            while True:
                camera.wait_recording(1)
        finally:
            camera.stop_recording()

```

4.9 快速采样与流化

继4.7 快速采样和处理之后，我们可以将视频采样技术与网络流相结合。服务器端脚本不会改变（它并不真正关心正在使用什么捕获技术——它只是从网络上读取 JPEG）。对客户端脚本的更改一开始可能很小：只需在capture_continuous()调用中将use_video_port设置为True：

```

import io
import socket
import struct
import time
import picamera

client_socket = socket.socket()
client_socket.connect(('my_server', 8000))
connection = client_socket.makefile('wb')

```

```

try:
    with picamera.PiCamera() as camera:
        camera.resolution = (640, 480)
        camera.framerate = 30
        time.sleep(2)
        start = time.time()
        count = 0
        stream = io.BytesIO()
        # Use the video-port for captures...
        for foo in camera.capture_continuous(stream, 'jpeg',
                                             use_video_port=True):
            connection.write(struct.pack('<L', stream.tell()))
            connection.flush()
            stream.seek(0)
            connection.write(stream.read())
            count += 1
            if time.time() - start > 30:
                break
            stream.seek(0)
            stream.truncate()
        connection.write(struct.pack('<L', 0))
finally:
    connection.close()
    client_socket.close()
    finish = time.time()
print('Sent %d images in %d seconds at %.2ffps' % (
    count, finish-start, count / (finish-start)))

```

使用这种技术，作者可以在 640x480 下管理大约 19fps 的流媒体。但是，利用快速捕获和处理中演示的 MJPEG 拆分，我们可以更快地进行管理：

```

import io
import socket
import struct
import time
import picamera

class SplitFrames(object):
    def __init__(self, connection):
        self.connection = connection
        self.stream = io.BytesIO()
        self.count = 0

```

```

def write(self, buf):
    if buf.startswith(b'\xff\xd8'):
        # Start of new frame; send the old one's length
        # then the data
        size = self.stream.tell()
        if size > 0:
            self.connection.write(struct.pack('<L', size))
            self.connection.flush()
            self.stream.seek(0)
            self.connection.write(self.stream.read(size))
            self.count += 1
            self.stream.seek(0)
        self.stream.write(buf)

client_socket = socket.socket()
client_socket.connect(('my_server', 8000))
connection = client_socket.makefile('wb')
try:
    output = SplitFrames(connection)
    with picamera.PiCamera(resolution='VGA', framerate=30) as
camera:
        time.sleep(2)
        start = time.time()
        camera.start_recording(output, format='mjpeg')
        camera.wait_recording(30)
        camera.stop_recording()
        # Write the terminating 0-length to the connection to let
the
        # server know we're done
        connection.write(struct.pack('<L', 0))
finally:
    connection.close()
    client_socket.close()
    finish = time.time()
print('Sent %d images in %d seconds at %.2ffps' % (
    output.count, finish-start, output.count / (finish-start)))

```

上面的脚本轻松实现了 30fps 的采样率。

4.10 网络流

通过网络流式传输视频非常复杂。在撰写本文时，仍然没有所有平台上的所有 Web 浏览器普遍支持的视频标准。此外，HTTP 最初被设计为用于服务网页的一次性协议。自从它发明以来，已经添加了各种附加功能以满足其不断增加的用例（文件下载、恢复、流媒体等），但事实仍然是目前没有“简单”的视频流解决方案。

如果您想播放“真实”视频格式（特别是 MPEG1），您可能需要查看 [pistreaming](#) 示例。但是，出于本节的目的，我们将使用更简单的格式：MJPEG。以下脚本使用 Python 的内置 `http.server` 模块制作一个简单的视频流服务器：

```
import io
import picamera
import logging
import socketserver
from threading import Condition
from http import server

PAGE="""\
<html>
<head>
<title>picamera MJPEG streaming demo</title>
</head>
<body>
<h1>PiCamera MJPEG Streaming Demo</h1>

</body>
</html>
"""

class StreamingOutput(object):
    def __init__(self):
        self.frame = None
        self.buffer = io.BytesIO()
        self.condition = Condition()

    def write(self, buf):
        if buf.startswith(b'\xff\xd8'):
            # New frame, copy the existing buffer's content and
            notify all
            # clients it's available
```

```

        self.buffer.truncate()
        with self.condition:
            self.frame = self.buffer.getvalue()
            self.condition.notify_all()
        self.buffer.seek(0)
    return self.buffer.write(buf)

class StreamingHandler(server.BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == '/':
            self.send_response(301)
            self.send_header('Location', '/index.html')
            self.end_headers()
        elif self.path == '/index.html':
            content = PAGE.encode('utf-8')
            self.send_response(200)
            self.send_header('Content-Type', 'text/html')
            self.send_header('Content-Length', len(content))
            self.end_headers()
            self.wfile.write(content)
        elif self.path == '/stream.mjpg':
            self.send_response(200)
            self.send_header('Age', 0)
            self.send_header('Cache-Control', 'no-cache, private')
            self.send_header('Pragma', 'no-cache')
            self.send_header('Content-Type', 'multipart/x-mixed-
replace; boundary=FRAME')
            self.end_headers()
            try:
                while True:
                    with output.condition:
                        output.condition.wait()
                        frame = output.frame
                    self.wfile.write(b'--FRAME\r\n')
                    self.send_header('Content-Type', 'image/jpeg')
                    self.send_header('Content-Length', len(frame))
                    self.end_headers()
                    self.wfile.write(frame)
                    self.wfile.write(b'\r\n')
            except Exception as e:
                logging.warning(
                    'Removed streaming client %s: %s',
                    self.client_address, str(e))

```

```

        else:
            self.send_error(404)
            self.end_headers()

class StreamingServer(socketserver.ThreadingMixIn,
server.HTTPServer):
    allow_reuse_address = True
    daemon_threads = True

with picamera.PiCamera(resolution='640x480', framerate=24) as
camera:
    output = StreamingOutput()
    camera.start_recording(output, format='mjpeg')
    try:
        address = ('', 8000)
        server = StreamingServer(address, StreamingHandler)
        server.serve_forever()
    finally:
        camera.stop_recording()

```

脚本运行后，可以使用网络浏览器访问 `http://your-pi-address:8000/` 以查看视频流。

Note

本例针对Python3.x，在Python2.x中，`http.server` 模块使用的是 `SimpleHTTPServer`

4.11 在录制的同时采样图像

相机能够在录制视频时捕捉静止图像。但是，如果尝试使用静态捕获模式进行此操作，则生成的视频将在静态图像捕获期间丢帧。这是因为通过静态端口捕获的图像需要更改模式，从而导致丢帧（这是在运行预览时捕获时看到的更高分辨率的闪烁）。

但是，如果 `use_video_port` 参数用于强制基于视频端口的图像捕获（请参阅快4.7 速捕获和处理），则不会发生模式更改，并且生成的视频不应有丢帧，假设图像可以在下一个视频帧到来前准备好：

```
import picamera

with picamera.PiCamera() as camera:
    camera.resolution = (800, 600)
    camera.start_preview()
    camera.start_recording('foo.h264')
    camera.wait_recording(10)
    camera.capture('foo.jpg', use_video_port=True)
    camera.wait_recording(10)
    camera.stop_recording()
```

上面的代码应该生成一个 20 秒的没有丢帧的视频，以及一个从 10 秒开始采样的静止帧。不过，更高的分辨率或非 JPEG 图像格式仍可能导致丢帧（只有 JPEG 编码是硬件加速的）。

4.12 以多个分辨率进行录像

通过使用视频分配器，摄像机能够同时录制不同分辨率的多个流。这对于对低分辨率流进行分析，同时记录高分辨率流以供存储或查看可能最有用。

以下简单示例演示了使用 `start_recording()` 方法的 `splitter_port` 参数开始两个同时录制，每个录制都具有不同的分辨率：

```
import picamera

with picamera.PiCamera() as camera:
    camera.resolution = (1024, 768)
    camera.framerate = 30
    camera.start_recording('highres.h264')
    camera.start_recording('lowres.h264', splitter_port=2, resize=(320, 240))
    camera.wait_recording(30)
    camera.stop_recording(splitter_port=2)
    camera.stop_recording()
```

总共有 4 个分路器端口可以使用（编号为 0、1、2 和 3）。录像方式默认使用分离器端口 1，而图像捕获方法默认使用分离器端口 0（当 `use_video_port` 参数也为 `True` 时）。分配器端口不能同时用于视频录制和图像捕获，因此建议您避免将分配器端口 0 用于视频录制，除非您不打算在录制时捕获图像。

1.3 版本新加入功能。

4.13 记录运动矢量数据

树莓派的摄像头能够输出运动矢量估计值，由摄像头的 H.264 编码器在生成压缩视频时计算得出。这些可以使用 `start_recording()` 方法的 `motion_output` 参数定向到单独的输出文件（或类似文件的对象）。像普通的输出参数一样，它接受一个表示文件名的字符串，或一个类似文件的对象：

```
import picamera

with picamera.PiCamera() as camera:
    camera.resolution = (640, 480)
    camera.framerate = 30
    camera.start_recording('motion.h264',
motion_output='motion.data')
    camera.wait_recording(10)
    camera.stop_recording()
```

运动数据是在宏块级别计算的（MPEG 宏块代表帧的 16x16 像素区域），并包括一系列额外的数据。因此，如果相机的分辨率为 640x480（如上例所示），则将有 41 列运动数据 $((640 \div 16) + 1)$ ，30 行 $(480 \div 16)$ 。

运动数据值为 4 字节长，由每个宏块的有符号 1 字节 x 向量、有符号 1 字节 y 向量和无符号 2 字节 SAD（绝对差和）值组成。因此，在上面的示例中，每帧将生成 4920 字节的运动数据 $(41 \times 30 \times 4)$ 。假设数据包含 300 帧（实际上可能包含更多帧），运动数据总共应为 1,476,000 字节。

以下代码演示了将运动数据加载到三维 numpy 数组中。第一个维度代表框架，后两个维度代表行，最后是列。数组使用结构化数据类型，允许轻松访问 x、y 和 SAD 值：

```
from __future__ import division

import numpy as np

width = 640
height = 480
cols = (width + 15) // 16
cols += 1 # there's always an extra column
rows = (height + 15) // 16

motion_data = np.fromfile(
```

```

    'motion.data', dtype=[
        ('x', 'i1'),
        ('y', 'i1'),
        ('sad', 'u2'),
    ])
frames = motion_data.shape[0] // (cols * rows)
motion_data = motion_data.reshape((frames, rows, cols))

# Access the data for the first frame
motion_data[0]

# Access just the x-vectors from the fifth frame
motion_data[4]['x']

# Access SAD values for the tenth frame
motion_data[9]['sad']

```

您可以简单地通过使用毕达哥拉斯定理计算矢量的幅度来计算矢量表示的运动量。SAD（绝对差异之和）值可用于确定编码器认为矢量代表原始参考帧的程度。

以下代码扩展了上面的示例，以使用 PIL 根据每帧运动矢量的幅度生成 PNG 图像：

```

from __future__ import division

import numpy as np
from PIL import Image

width = 640
height = 480
cols = (width + 15) // 16
cols += 1
rows = (height + 15) // 16

m = np.fromfile(
    'motion.data', dtype=[
        ('x', 'i1'),
        ('y', 'i1'),
        ('sad', 'u2'),
    ])
frames = m.shape[0] // (cols * rows)
m = m.reshape((frames, rows, cols))

```

```

for frame in range(frames):
    data = np.sqrt(
        np.square(m[frame]['x'].astype(np.float)) +
        np.square(m[frame]['y'].astype(np.float))
    ).clip(0, 255).astype(np.uint8)
    img = Image.fromarray(data)
    filename = 'frame%03d.png' % frame
    print('Writing %s' % filename)
    img.save(filename)

```

您可能希望使用 `picamera.array` 模块中的 `PiMotionArray` 和 `PiMotionAnalysis` 类，它将上述配方简化为以下内容：

```

import numpy as np
import picamera
import picamera.array
from PIL import Image

with picamera.PiCamera() as camera:
    with picamera.array.PiMotionArray(camera) as stream:
        camera.resolution = (640, 480)
        camera.framerate = 30
        camera.start_recording('/dev/null', format='h264',
motion_output=stream)
        camera.wait_recording(10)
        camera.stop_recording()
        for frame in range(stream.array.shape[0]):
            data = np.sqrt(
                np.square(stream.array[frame]['x'].astype(np.float))
+
                np.square(stream.array[frame]['y'].astype(np.float))
            ).clip(0, 255).astype(np.uint8)
            img = Image.fromarray(data)
            filename = 'frame%03d.png' % frame
            print('Writing %s' % filename)
            img.save(filename)

```

以下命令行可用于使用 `ffmpeg` 从生成的 PNG 生成动画（这将在树莓派上花费很长时间，因此您可能希望将图像传输到更快的机器以进行此步骤）：

```

avconv -r 30 -i frame%03d.png -filter:v scale=640:480 -c:v libx264
motion.mp4

```

最后，为了演示运动矢量可以完成什么，这里有一个手势检测系统的代码：

```
import os
import numpy as np
import picamera
from picamera.array import PiMotionAnalysis

class GestureDetector(PiMotionAnalysis):
    QUEUE_SIZE = 10 # the number of consecutive frames to analyze
    THRESHOLD = 4.0 # the minimum average motion required in either
axis

    def __init__(self, camera):
        super(GestureDetector, self).__init__(camera)
        self.x_queue = np.zeros(self.QUEUE_SIZE, dtype=np.float)
        self.y_queue = np.zeros(self.QUEUE_SIZE, dtype=np.float)
        self.last_move = ''

    def analyze(self, a):
        # Roll the queues and overwrite the first element with a new
        # mean (equivalent to pop and append, but faster)
        self.x_queue[1:] = self.x_queue[:-1]
        self.y_queue[1:] = self.y_queue[:-1]
        self.x_queue[0] = a['x'].mean()
        self.y_queue[0] = a['y'].mean()
        # Calculate the mean of both queues
        x_mean = self.x_queue.mean()
        y_mean = self.y_queue.mean()
        # Convert left/up to -1, right/down to 1, and movement below
        # the threshold to 0
        x_move = (
            '' if abs(x_mean) < self.THRESHOLD else
            'left' if x_mean < 0.0 else
            'right')
        y_move = (
            '' if abs(y_mean) < self.THRESHOLD else
            'down' if y_mean < 0.0 else
            'up')
        # Update the display
        movement = ('%s %s' % (x_move, y_move)).strip()
        if movement != self.last_move:
            self.last_move = movement
            if movement:
```



```

        print(movement)

with picamera.PiCamera(resolution='VGA', framerate=24) as camera:
    with GestureDetector(camera) as detector:
        camera.start_recording(
            os.devnull, format='h264', motion_output=detector)
        try:
            while True:
                camera.wait_recording(1)
        finally:
            camera.stop_recording()

```

在距离摄像头几英寸的范围内，与摄像头平行地上下左右移动您的手，您应该会看到控制台上显示的方向。

1.5版本中新加入的功能。

4.14 从循环流中分离视频

这个例子建立在记录到循环流中的一个和记录时捕获图像中的一个，以演示安全应用程序的开始。和以前一样，PiCameraCircularIO实例用于将视频的最后几秒记录在内存中。在录制视频时，会进行基于视频端口的静态捕获，以提供具有某些输入的运动检测例程（实际的运动检测算法留给读者作为练习）。

一旦检测到运动，最后 10 秒的视频将写入磁盘，并将视频录制拆分到另一个磁盘文件以继续进行，直到不再检测到运动。一旦不再检测到运动，我们将记录拆分回内存中的环形缓冲区：

```

import io
import random
import picamera
from PIL import Image

prior_image = None

def detect_motion(camera):
    global prior_image
    stream = io.BytesIO()
    camera.capture(stream, format='jpeg', use_video_port=True)
    stream.seek(0)
    if prior_image is None:
        prior_image = Image.open(stream)
    return False

```

```

else:
    current_image = Image.open(stream)
    # Compare current_image to prior_image to detect motion.
This is
    # left as an exercise for the reader!
    result = random.randint(0, 10) == 0
    # Once motion detection is done, make the prior image the
current
    prior_image = current_image
    return result

with picamera.PiCamera() as camera:
    camera.resolution = (1280, 720)
    stream = picamera.PiCameraCircularIO(camera, seconds=10)
    camera.start_recording(stream, format='h264')
    try:
        while True:
            camera.wait_recording(1)
            if detect_motion(camera):
                print('Motion detected!')
                # As soon as we detect motion, split the recording
to
                # record the frames "after" motion
                camera.split_recording('after.h264')
                # Write the 10 seconds "before" motion to disk as
well
                stream.copy_to('before.h264', seconds=10)
                stream.clear()
                # Wait until motion is no longer detected, then
split
                # recording back to the in-memory circular buffer
                while detect_motion(camera):
                    camera.wait_recording(1)
                    print('Motion stopped!')
                    camera.split_recording(stream)
            finally:
                camera.stop_recording()

```

此示例还演示了使用`copy_to()`方法的`seconds`参数将前文件限制为 10 秒的数据（假设循环缓冲区可能包含的数据远不止于此）。

1.0版本新加入功能。

1.1版本新加入`copy_to()`功能。

4.15 自定义编码器

您可以覆盖和/或扩展在图像或视频捕获期间使用的编码器类。这对于视频捕获特别有用，因为它允许您运行自己的代码以响应每一帧，尽管自然而然，在编码器的回调中运行的任何代码都必须相当快，以避免停止编码器管道。

编写自定义编码器比编写自定义输出要困难得多，而且在大多数情况下几乎没有什么好处。自定义编码器唯一为您提供的自定义输出无法访问缓冲区标头标志。对于许多输出格式（例如 MJPEG 和 YUV），这些不会告诉您任何有趣的信息（即它们只会指示缓冲区包含一个完整的帧而没有其他内容）。目前，缓冲区标头标志包含有用信息的唯一格式是 H.264。即便如此，大部分信息（I 帧、P 帧、运动信息等）都可以从 `frame` 属性访问，您可以从自定义输出的 `write` 方法访问该属性。

`picamera` 定义的编码器类形成以下层次结构（暗类实际上由 `picamera` 中的实现实例化，轻类实现基本功能但在技术上不是“抽象的”）：

下表详细说明了 `PiCamera` 方法对应的编码器类，以及它们调用哪些方法来构造这些编码器：

Method(s)	Calls	Returns
<code>capture()</code> <code>capture_continuous()</code> <code>capture_sequence()</code>	<code>_get_image_encoder()</code>	<code>PiCookedOneImageEncoder</code> <code>PiRawOneImageEncoder</code>
<code>capture_sequence()</code>	<code>_get_image_encoder()</code>	<code>PiCookedMultiImageEncoder</code> <code>PiRawMultiImageEncoder</code>
<code>start_recording()</code> <code>record_sequence()</code>	<code>_get_video_encoder()</code>	<code>PiCookedVideoEncoder</code> <code>PiRawVideoEncoder</code>

在图像编码器类的情况下，我们建议您熟悉这些类的特定功能，以便您可以确定要扩展的最佳类以满足您的特定需求。您可能会发现其中一个中间类是您自己修改的更好基础。

在以下示例配方中，我们将扩展 `PiCookedVideoEncoder` 类以存储捕获的 I 帧和 P 帧的数量（相机的编码器不使用 B 帧）：

```
import picamera
import picamera.mmal as mmal

# Override PiVideoEncoder to keep track of the number of each type
of frame
```

```

class MyEncoder(picamera.PiCookedVideoEncoder):
    def start(self, output, motion_output=None):
        self.parent.i_frames = 0
        self.parent.p_frames = 0
        super(MyEncoder, self).start(output, motion_output)

    def _callback_write(self, buf):
        # Only count when buffer indicates it's the end of a frame,
and
        # it's not an SPS/PPS header (..._CONFIG)
        if (
            (buf.flags & mmal.MMAL_BUFFER_HEADER_FLAG_FRAME_END)
and
            not (buf.flags &
mmal.MMAL_BUFFER_HEADER_FLAG_CONFIG)
        ):
            if buf.flags & mmal.MMAL_BUFFER_HEADER_FLAG_KEYFRAME:
                self.parent.i_frames += 1
            else:
                self.parent.p_frames += 1
        # Remember to return the result of the parent method!
        return super(MyEncoder, self)._callback_write(buf)

# Override PiCamera to use our custom encoder for video recording
class MyCamera(picamera.PiCamera):
    def __init__(self):
        super(MyCamera, self).__init__()
        self.i_frames = 0
        self.p_frames = 0

    def _get_video_encoder(
        self, camera_port, output_port, format, resize,
**options):
        return MyEncoder(
            self, camera_port, output_port, format, resize,
**options)

with MyCamera() as camera:
    camera.start_recording('foo.h264')
    camera.wait_recording(10)
    camera.stop_recording()
    print('Recording contains %d I-frames and %d P-frames' % (
        camera.i_frames, camera.p_frames))

```

不过请注意，上述方法存在缺陷：PiCamera 能够启动多个同时录制。如果这与上述配方一起使用，那么每个编码器最终都会增加 MyCamera 实例上的 `i_frames` 和 `p_frames` 属性，从而导致不正确的结果。

1.5版本新加入的功能。

4.16 采集原始拜耳数据

`capture()` 方法的 `bayer` 参数可以使相机传感器记录的原始 Bayer 数据作为图像元数据的一部分输出。

Note

`bayer` 参数仅适用于 JPEG 格式，并且仅用于从静止端口捕获（即当 `use_video_port` 为 `False` 时，默认情况下）。

原始拜耳数据与简单的未编码捕获有很大不同；它是在任何 GPU 处理（包括自动白平衡、晕影补偿、平滑、缩小等）之前由相机传感器记录的数据。这也意味着：

- 无论相机的输出分辨率和任何调整大小参数如何，拜耳数据始终为全分辨率。
- Bayer 数据占用 V1 模块输出文件的最后 6,404,096 字节，或 V2 模块的最后 10,270,208 字节。它的前 32,768 个字节是以字符串“BRCM”开头的标题数据。
- Bayer 数据由 10 位值组成，因为这是树莓派相机模块中使用的 OV5647 和 IMX219 传感器模组。10 位值被组织为 4 个 8 位值，然后是 4 个值的低位 2 位打包成第五个字节。
- 拜耳数据以 BGGR 模式组织（普通拜耳 CFA 的微小变化）。因此，原始数据的绿色像素是红色或蓝色像素的两倍，如果查看“原始数据”会显得非常奇怪（太暗、太绿，并且沿任何直边都有拉链效应）。
- 要从原始拜耳数据制作“正常”外观的图像，您至少需要执行边缘平滑去马赛克，可能还需要某种形式的色彩平衡。

该示例脚本（含大量注释）使相机捕获包含原始拜耳数据的图像。然后继续将 Bayer 数据解包到表示原始 RGB 数据的 3 维 numpy 数组中，最后使用加权平均值执行基本的去马赛克步骤。一些 numpy 技巧用于提高性能，但请记住，所有处理都在 CPU 上进行，并且会比正常图像捕获慢得多：

```

from __future__ import (
    unicode_literals,
    absolute_import,
    print_function,
    division,
)

import io
import time
import picamera
import numpy as np
from numpy.lib.stride_tricks import as_strided

stream = io.BytesIO()
with picamera.PiCamera() as camera:
    # Let the camera warm up for a couple of seconds
    time.sleep(2)
    # Capture the image, including the Bayer data
    camera.capture(stream, format='jpeg', bayer=True)
    ver = {
        'RP_ov5647': 1,
        'RP_imx219': 2,
    }[camera.exif_tags['IFD0.Model']]

# Extract the raw Bayer data from the end of the stream, check the
# header and strip it off before converting the data into a numpy
array

offset = {
    1: 6404096,
    2: 10270208,
}[ver]
data = stream.getvalue()[offset:]
assert data[:4] == 'BRCM'
data = data[32768:]
data = np.fromstring(data, dtype=np.uint8)

# For the V1 module, the data consists of 1952 rows of 3264 bytes of
data.
# The last 8 rows of data are unused (they only exist because the
maximum
# resolution of 1944 rows is rounded up to the nearest 16).

```

```

#
# For the V2 module, the data consists of 2480 rows of 4128 bytes of
data.
# There's actually 2464 rows of data, but the sensor's raw size is
2466
# rows, rounded up to the nearest multiple of 16: 2480.
#
# Likewise, the last few bytes of each row are unused (why?). Here
we
# reshape the data and strip off the unused bytes.

reshape, crop = {
    1: ((1952, 3264), (1944, 3240)),
    2: ((2480, 4128), (2464, 4100)),
}[ver]
data = data.reshape(reshape)[:crop[0], :crop[1]]

# Horizontally, each row consists of 10-bit values. Every four bytes
are
# the high 8-bits of four values, and the 5th byte contains the
packed low
# 2-bits of the preceding four values. In other words, the bits of
the
# values A, B, C, D and arranged like so:
#
# byte 1   byte 2   byte 3   byte 4   byte 5
# AAAAAAAA BBBBBBBB CCCCCCCC DDDDDDDD AABBCDD
#
# Here, we convert our data into a 16-bit array, shift all values
left by
# 2-bits and unpack the low-order bits from every 5th byte in each
row,
# then remove the columns containing the packed bits

data = data.astype(np.uint16) << 2
for byte in range(4):
    data[:, byte::5] |= ((data[:, 4::5] >> ((4 - byte) * 2)) & 0b11)
data = np.delete(data, np.s_[4::5], 1)

# Now to split the data up into its red, green, and blue components.
The
# Bayer pattern of the OV5647 sensor is BGGR. In other words the
first

```

```

# row contains alternating green/blue elements, the second row
contains
# alternating red/green elements, and so on as illustrated below:
#
# GBGBGBGBGBGBGB
# RGRGRGRGRGRGRG
# GBGBGBGBGBGBGB
# RGRGRGRGRGRGRG
#
# Please note that if you use vflip or hflip to change the
orientation
# of the capture, you must flip the Bayer pattern accordingly

rgb = np.zeros(data.shape + (3,), dtype=data.dtype)
rgb[1::2, 0::2, 0] = data[1::2, 0::2] # Red
rgb[0::2, 0::2, 1] = data[0::2, 0::2] # Green
rgb[1::2, 1::2, 1] = data[1::2, 1::2] # Green
rgb[0::2, 1::2, 2] = data[0::2, 1::2] # Blue

# At this point we now have the raw Bayer data with the correct
values
# and colors but the data still requires de-mosaicing and
# post-processing. If you wish to do this yourself, end the script
here!
#
# Below we present a fairly naive de-mosaic method that simply
# calculates the weighted average of a pixel based on the pixels
# surrounding it. The weighting is provided by a byte representation
of
# the Bayer filter which we construct first:

bayer = np.zeros(rgb.shape, dtype=np.uint8)
bayer[1::2, 0::2, 0] = 1 # Red
bayer[0::2, 0::2, 1] = 1 # Green
bayer[1::2, 1::2, 1] = 1 # Green
bayer[0::2, 1::2, 2] = 1 # Blue

# Allocate an array to hold our output with the same shape as the
input
# data. After this we define the size of window that will be used to
# calculate each weighted average (3x3). Then we pad out the rgb and
# bayer arrays, adding blank pixels at their edges to compensate for
the

```



```

# size of the window when calculating averages for edge pixels.

output = np.empty(rgb.shape, dtype=rgb.dtype)
window = (3, 3)
borders = (window[0] - 1, window[1] - 1)
border = (borders[0] // 2, borders[1] // 2)

rgb = np.pad(rgb, [
    (border[0], border[0]),
    (border[1], border[1]),
    (0, 0),
], 'constant')
bayer = np.pad(bayer, [
    (border[0], border[0]),
    (border[1], border[1]),
    (0, 0),
], 'constant')

# For each plane in the RGB data, we use a nifty numpy trick
# (as_strided) to construct a view over the plane of 3x3 matrices.
# We do
# the same for the bayer array, then use Einstein summation on each
# (np.sum is simpler, but copies the data so it's slower), and
# divide
# the results to get our weighted average:

for plane in range(3):
    p = rgb[..., plane]
    b = bayer[..., plane]
    pview = as_strided(p, shape=(
        p.shape[0] - borders[0],
        p.shape[1] - borders[1]) + window, strides=p.strides * 2)
    bview = as_strided(b, shape=(
        b.shape[0] - borders[0],
        b.shape[1] - borders[1]) + window, strides=b.strides * 2)
    psum = np.einsum('ijkl->ij', pview)
    bsum = np.einsum('ijkl->ij', bview)
    output[..., plane] = psum // bsum

# At this point output should contain a reasonably "normal" looking
# image, although it still won't look as good as the camera's normal
# output (as it lacks vignette compensation, AWB, etc).
#

```

```
# If you want to view this in most packages (like GIMP) you'll need
to
# convert it to 8-bit RGB data. The simplest way to do this is by
# right-shifting everything by 2-bits (yes, this makes all that
# unpacking work at the start rather redundant...)

output = (output >> 2).astype(np.uint8)
with open('image.data', 'wb') as f:
    output.tofile(f)
```

本示例的增强版本（也处理翻转和旋转引起的不同拜耳顺序）也封装在picamera.array模块的PiBayerArray类中，这意味着同样可以实现如下：

```
import time
import picamera
import picamera.array
import numpy as np

with picamera.PiCamera() as camera:
    with picamera.array.PiBayerArray(camera) as stream:
        camera.capture(stream, 'jpeg', bayer=True)
        # Demosaic data and write to output (just use stream.array
if you
        # want to skip the demosaic step)
        output = (stream.demosaic() >> 2).astype(np.uint8)
        with open('image.data', 'wb') as f:
            output.tofile(f)
```

1.3版本新加入功能。

1.5版本新加入picamera.array模块。

4.17 使用闪光灯

树莓派相机模块包括一个 LED 闪光灯驱动器，可用于在拍摄时照亮场景。闪光灯驱动程序有两个可配置的 GPIO 引脚：

- 一种用于连接基于 LED 的闪光灯（氙气闪光灯无法与相机模块配合使用，因为它具有滚动快门）。这将在（闪光测光）之前和拍摄期间触发
- 一个用于可选的隐私指示器（某些司法管辖区对相机的要求）。拍照后会触发，表示相机已被使用

这些引脚是通过更新 VideoCore 设备树 blob 来配置的。首先，安装设备树编译器，然后获取默认设备树源的副本：

```
$ sudo apt-get install device-tree-compiler
$ wget https://github.com/raspberrypi/firmware/raw/master/extra/dt-blob.dts
```

设备树源包含许多用大括号括起来的部分，这些部分形成了定义的层次结构。要编辑的部分将取决于您拥有的树莓派版本（如果不确定，请检查板上的PCB印刷版本号）：

Model	Section
Raspberry Pi Model B rev 1	/videocore/pins_rev1
Raspberry Pi Model A and Model B rev 2	/videocore/pins_rev2
Raspberry Pi Model A+	/videocore/pins_aplus
Raspberry Pi Model B+ rev 1.1	/videocore/pins_bplus1
Raspberry Pi Model B+ rev 1.2	/videocore/pins_bplus2
Raspberry Pi 2 Model B rev 1.0	/videocore/pins_2b1
Raspberry Pi 2 Model B rev 1.1 and rev 1.2	/videocore/pins_2b2
Raspberry Pi 3 Model B rev 1.0	/videocore/pins_3b1
Raspberry Pi 3 Model B rev 1.2	/videocore/pins_3b2
Raspberry Pi Zero rev 1.2 and rev 1.3	/videocore/pins_pi0

在您的树莓派硬件支持中，您将找到pin_config和pin_defines部分。设置pin_config时，您需要将要用于闪光灯和隐私指示器的 GPIO 引脚配置为使用下拉终止。然后，在pin_defines部分下，您需要将这些引脚与FLASH_O_ENABLE和FLASH_O_INDICATOR引脚相关联。

例如，要将 GPIO 17 配置为闪存引脚，而没有隐私指示器引脚，在树莓派 2 Model B rev 1.1 上，您需要在/videocore/pins_2b2/pin_config部分下添加以下行：

```
pin@p17 { function = "output"; termination = "pull_down"; };
```

请注意，GPIO 引脚将根据 Broadcom 引脚编号（RPI.GPIO 库中的 BCM 模式，而不是 BOARD 模式）进行编号。然后更改/videocore/pins_2b2/pin_defines下的以下部分。具体来说，将类型从“absent”更改为“internal”，并添加将 flash pin 定义为 GPIO 17 的 number 属性：

```
pin_define@FLASH_0_ENABLE {  
    type = "internal";  
    number = <17>;  
};
```

更新设备树源后，您现在需要将其编译为二进制 blob 以供固件读取。

```
$ dtc -q -I dts -O dtb dt-blob.dts -o dt-blob.bin
```

该命令具有如下组件：

- dtc - 执行设备树编译器
- -I dts - 输入文件为设备树源格式
- -O dtb - 输出文件应以设备树二进制格式生成
- dt-blob.dts - 第一个匿名参数是输入文件名
- -o dt-blob.bin - 输出文件名

这应该什么都不输出。如果你收到很多警告，你就忘记了 -q 开关；您可以忽略警告。如果输出任何其他内容，则很可能是一条错误消息，表明您在设备树源中犯了错误。在这种情况下，请仔细检查您的编辑（请注意，例如，部分和属性必须以分号结尾），然后重试。

现在设备树binary blob已经制作完成，需要放在SD卡的第一个分区上。在非Raspbian安装的情况下，通常挂载在 /boot 分区：

```
$ sudo cp dt-blob.bin /boot/
```

但是，在非Raspbian系统安装的情况下，这是恢复分区，默认情况下不挂载：

```
$ sudo mkdir /mnt/recovery  
$ sudo mount /dev/mmcblk0p1 /mnt/recovery  
$ sudo cp dt-blob.bin /mnt/recovery  
$ sudo umount /mnt/recovery  
$ sudo rmdir /mnt/recovery
```

请注意文件名和位置很重要。二进制blob必须命名为dt-blob.bin（全部小写），并且必须放在SD卡第一个分区的根目录下。重新启动树莓派（以激活新的设备树配置）后，您可以使用以下简单脚本测试闪光灯：

```
import picamera  
  
with picamera.PiCamera() as camera:  
    camera.flash_mode = 'on'  
    camera.capture('foo.jpg')
```

在脚本执行期间，您应该会看到闪光灯 LED 闪烁两次。

Warning

GPIO 仅具有有限的电流驱动，不足以为通常用作手机闪光灯的那种 LED 供电。您将需要合适的驱动电路来为此类设备供电，否则可能会损坏您的树莓派硬件。

树莓派论坛上的一位开发人员指出：作为参考，我们在手机上使用的闪存驱动芯片往往会驱动高达 500mA 的电流进入 LED。所以请使用大电流电源。

如果您希望在不将任何东西连接到 GPIO 引脚的情况下试用闪光灯驱动程序，您还可以重新配置相机自己的 LED 以充当闪光灯 LED。显然这对实际的闪光摄影没有好处，但它可以证明您的配置是否良好。在这种情况下，您无需向 `pin_config` 部分添加任何内容（相机的 LED 引脚已定义为使用下拉终止），但您需要将 `CAMERA_O_LED` 设置为不存在，并将 `FLASH_O_ENABLE` 设置为旧的 `CAMERA_O_LED` 定义（这将是引脚 5 在 `pin_rev1` 和 `pins_rev2` 的情况下，以及其他所有情况下的 `pin 32`）。进行如下更改，将：

```
pin_define@CAMERA_O_LED {
    type = "internal";
    number = <5>;
};
pin_define@FLASH_O_ENABLE {
    type = "absent";
};
```

替换为：

```
pin_define@CAMERA_O_LED {
    type = "absent";
};
pin_define@FLASH_O_ENABLE {
    type = "internal";
    number = <5>;
};
```

根据上述说明编译和安装设备树 blob 并重新启动树莓派后，您应该会发现摄像头闪光灯现在可以由上述 Python 脚本驱动。

1.10 版中的新功能。

五、常见问题（FAQ）

5.1 `AttributeError: 'module' 对象没有属性 'PiCamera'`

您已将脚本命名为 `picamera.py`（或将其他脚本命名为 `picamera.py`。如果以系统或第三方包命名脚本，则会中断该系统或第三方包的导入。删除或重命名该脚本（以及任何关联的 `.pyc` 文件），然后重试。

5.2 我可以将预览放在窗口中吗？

不。相机模块的预览系统非常粗糙：它只是告诉 GPU 在树莓派的视频输出上叠加预览。预览不知道（或与 X-Windows 环境交互）（顺便说一句，这就是为什么即使没有任何人登录，预览也可以从命令行非常愉快地工作）。

也就是说，可以通过预览对象的 `window` 属性调整预览区域的大小和重新定位。如果您的程序可以响应窗口重新定位和调整大小事件，您可以“欺骗”并将预览定位在目标窗口的边界内。然而，目前没有办法让任何东西出现在预览之上，所以这充其量是一个不完美的解决方案。

5.3 我开始预览但看不到我的控制台！

如上所述，预览只是覆盖在树莓派的视频输出上。如果您开始预览，您可能会发现您再也看不到您的控制台，并且没有明显的方法可以将其恢复。如果您对自己的打字技巧有信心，可以尝试通过在隐藏控制台中“盲目地”输入来调用 `stop_preview()`。但是，恢复显示的最简单方法通常是按 `Ctrl+D` 终止 Python 进程（这也应该关闭相机）。

开始预览时，您可能希望将 `start_preview()` 方法的 `alpha` 参数设置为 128 之类的值。这应该确保在显示预览时，它是部分透明的，因此您仍然可以看到您的控制台。

5.4 预览在我的 PiTFT 屏幕上不起作用

相机的预览系统直接覆盖在 HDMI 或复合视频端口上的树莓派输出。此时，它不会与 PiTFT 等 GPIO 驱动的显示器一起运行。一些项目，如 Adafruit 触摸屏相机项目，通过快速捕获未编码的图像并将它们显示在 PiTFT 上来近似预览。

5.5 相机需要多大功率？

相机在运行时需要250mA电流。请注意，简单地创建 PiCamera 对象意味着相机正在运行（由于隐藏预览已启动以允许自动曝光算法运行）。如果您使用电池运行您的树莓派，您应该在不需要相机时`close()`（或销毁）实例以节省电量。例如，以下代码在一个小时内捕获了 60 张图像，但让相机一直运行：

```
import picamera
import time

with picamera.PiCamera() as camera:
    camera.resolution = (1280, 720)
    time.sleep(1) # Camera warm-up time
    for i, filename in
enumerate(camera.capture_continuous('image{counter:02d}.jpg')):
        print('Captured %s' % filename)
        # Capture one image a minute
        time.sleep(60)
        if i == 59:
            break
```

相比之下，此代码在两次拍摄之间关闭相机（但因此无法使用方便的`capture_continuous()`方法）：

```
import picamera
import time

for i in range(60):
    with picamera.PiCamera() as camera:
        camera.resolution = (1280, 720)
        time.sleep(1) # Camera warm-up time
        filename = 'image%02d.jpg' % i
        camera.capture(filename)
        print('Captured %s' % filename)
    # Capture one image a minute
    time.sleep(59)
```

Note

请注意上述脚本中的时间是近似值。3.6 定时拍摄中给出了更精确的计时示例。

如果您在相机处于活动状态时遇到锁定或重新启动，则您的电源可能不足。运行带有活动相机模块的树莓派的实际电流最小值为 1A；如果附加了额外的外围设备，则可能需要更多。

5.6 如何以相同的设置连续拍摄两张照片？

请参阅3.5采样图像的一致性。

5.7 我可以将 picamera 与 USB 网络摄像头一起使用吗？

不可以。picamera 库依赖于特定于树莓派的相机模块的 libmmal。

5.8 我如何知道我安装了哪个版本的 picamera？

picamera 库依赖 setuptools 包进行安装服务。您可以使用setuptools pkg_resources API来查询 Python 环境中可用的 picamera 版本，如下所示：

```
>>> from pkg_resources import require
>>> require('picamera')
[picamera 1.2 (/usr/local/lib/python2.7/dist-packages)]
>>> require('picamera')[0].version
'1.2'
```

如果您安装了多个版本（例如来自pip和apt-get），它们将不会出现在require方法返回的列表中。但是，列表中的第一个条目将是import picamera将导入的版本。

如果您收到错误“No module named pkg_resources”，则需要安装pip实用程序。这可以在 Raspbian 中使用以下命令完成：

```
$ sudo apt-get install python-pip
```

5.9 为什么我无法升级到最新版本？

如果您使用的是 Raspbian，请首先检查您是否同时安装了 PyPI (pip) 和 apt (apt-get)。（有就可以，不一定需要最新版本）

其次，请理解，虽然 PyPI 的发布过程是完全自动化的（所以一旦发布了新的 picamera 版本，它就会在 PyPI 上可用），但 Raspbian 包的发布过程是半手动的。在 Raspbian 存储库中可以访问更新的 picamera 软件包之前，通常会延迟几天。

急于尝试最新版本的用户可以选择卸载他们基于apt的副本（安装说明中提供了卸载说明，并使用 pip 进行安装。但是，请注意，保持基于 PyPI 的安装是最新的是一个更加手动过程（坚持使用apt可确保使用简单的 `sudo apt-get upgrade` 命令升级所有内容）。

5.10 为什么流式传输视频时会有如此多的延迟？

首先要了解的是，流延迟与事物的编码或发送端（树莓派）几乎没有关系，而与播放端或接收端的关系更大。如果树莓派无法在下一帧到达之前对帧进行编码，则它根本无法录制视频（因为其内部缓冲区会迅速被未编码的帧填满）。

那么，为什么玩家通常会引入几秒钟的延迟呢？主要原因是大多数播放器（例如 VLC）都针对通过网络播放流进行了优化。这些播放器会分配一个大的（多秒）缓冲区，并且只有在填满后才开始播放，以防止将来可能出现的数据包丢失。

所有此类播放器至少分配几帧值得缓冲的第二个原因是 MPEG 标准包括某些需要它的帧类型：

- I 帧（帧内，也称为“关键帧”）。这些帧包含完整的图片，因此是最大的一类帧。它们发生在播放开始时和流期间的周期性点。
- P 帧（预测帧）。这些帧描述了从前一帧到当前帧的变化，因此必须成功解码前一帧才能解码 P 帧。
- B 帧（双向预测帧）。这些帧描述了从下一帧到当前帧的变化，因此必须成功解码下一帧才能解码当前 B 帧。

B 帧不是由树莓派的相机（或者，据我所知，大多数实时记录相机）产生的，因为它需要在对当前帧进行编码之前缓冲尚未录制的帧。但是，大多数录制媒体（DVD、蓝光以及网络视频流）都使用它们，因此播放器必须支持它们。编写这样的播放器是最简单的，假设任何源都可能包含 B 帧，并始终缓冲至少 2 帧的数据以使解码更简单。

至于介于两者之间的网络，慢速 wifi 网络可能会引入一帧的延迟，但仅此而已。检查整个网络的 ping 时间；它很可能小于 30 毫秒，在这种情况下，您的网络无法考虑超过一帧的延迟。

TL;DR：流视频时出现大量延迟的原因与树莓派无关。您需要让您的视频播放器减少或放弃缓冲。

5.11 为什么循环缓冲区中有超过20秒的视频？

阅读记录底部的注释到循环流示例。当您设置循环流的秒数时，您是在设置给定比特率的下限（默认为 17Mbps - 与视频录制默认值相同）。如果录制的场景具有低运动或复杂性，则流可以存储比指定秒数多得多的时间。

如果您需要从流中复制特定秒数，请参阅`copy_to()`方法（在 1.11 版中引入）的 `seconds` 参数。

最后，如果您为流和录制指定不同的比特率限制，秒限制将不准确。

5.12 我可以移动注释文本吗？

否：固件不提供移动注释文本的方法。注释的唯一可配置属性是当前的颜色和字体大小。

5.13 为什么在 VLC/omxplayer/etc. 中播放太快/太慢？

相机的 H264 编码器不会输出完整的 MP4 文件（其中包含每秒帧数的元数据）。相反，它输出一个 H264 NAL 流，其中只有帧大小和一些其他细节（但不是 FPS）。

大多数播放器（如 VLC）默认为 24、25 或 30 fps。因此，12fps 的录制会显得“快”，而 60fps 的录制会显得“慢”。您的播放客户端需要被告知播放时使用的 fps（假设它支持这样的选项）。

对于那些想知道为什么相机不输出完整 MP4 文件的人，请考虑一下 Pi 相机的传统是手机相机。在这些设备中，您只希望相机输出 H264 流，以便您可以将其与从麦克风输入记录的 AAC 流进行多路复用，并将结果包装成完整的 MP4 文件。

要将 H264 NAL 流转换为完整的 MP4 文件，有几个选项。最简单的方法是使用 Raspbian 上 `gpac` 包中的 `MP4Box` 实用程序。不幸的是，这只适用于文件；它不能接受重定向的流：

```
$ sudo apt-get install gpac
...
$ MP4Box -add input.h264 output.mp4
```

或者，您可以使用 VLC 的控制台版本来处理转换。这是一个更复杂的命令行，但功能更强大（它将处理重定向的流，并且可以用于包括 HTTP、RTP 等在内的大量输出）：

```
$ sudo apt-get install vlc
...
$ cvlc input.h264 --play-and-exit --sout \
> '#standard{access=file,mux=mp4,dst=output.mp4}' :demux=h264 \
```

或者从标准输入读取：

```
$ raspivid -t 5000 -o - | cvlc stream:///dev/stdin \
> --play-and-exit --sout \
> '#standard{access=file,mux=mp4,dst=output.mp4}' :demux=h264 \
```

5.14 V2 模块上的全分辨率资源不足

请参阅6.3 硬件限制。

5.15 在 V2 模块上以全分辨率预览闪烁

使用新的`resolution`为预览选择较低的分辨率，或在开始预览时指定一个分辨率。 例如：

```
from picamera import PiCamera

camera = PiCamera()
camera.resolution = camera.MAX_RESOLUTION
camera.start_preview(resolution=(1024, 768))
```

5.16 相机锁定多处理

相机固件设计为一次由一个进程使用。 尝试同时从多个进程使用相机会以多种方式失败（从简单的错误到进程锁定）。

Python 的`multiprocessing`模块为并行处理的目的创建 Python 进程的多个副本（通常通过`os.fork()`）。 虽然您可以对 `picamera` 使用多处理，但您必须确保在任何给定时间只有一个进程创建`PiCamera`实例。

以下脚本演示了一种方法，其中一个进程拥有相机，该进程通过`Queue`处理将捕获的帧传播到其他进程：

```
import os
import io
import time
import multiprocessing as mp
from queue import Empty
import picamera
```

```

from PIL import Image

class QueueOutput(object):
    def __init__(self, queue, finished):
        self.queue = queue
        self.finished = finished
        self.stream = io.BytesIO()

    def write(self, buf):
        if buf.startswith(b'\xff\xd8'):
            # New frame, put the last frame's data in the queue
            size = self.stream.tell()
            if size:
                self.stream.seek(0)
                self.queue.put(self.stream.read(size))
                self.stream.seek(0)
            self.stream.write(buf)

    def flush(self):
        self.queue.close()
        self.queue.join_thread()
        self.finished.set()

def do_capture(queue, finished):
    with picamera.PiCamera(resolution='VGA', framerate=30) as camera:
        output = QueueOutput(queue, finished)
        camera.start_recording(output, format='mjpeg')
        camera.wait_recording(10)
        camera.stop_recording()

def do_processing(queue, finished):
    while not finished.wait(0.1):
        try:
            stream = io.BytesIO(queue.get(False))
        except Empty:
            pass
        else:
            stream.seek(0)
            image = Image.open(stream)
            # Pretend it takes 0.1 seconds to process the frame; on
a quad-core
            # Pi this gives a maximum processing throughput of 40fps

```

```

        time.sleep(0.1)
        print('%d: Processing image with size %dx%d' % (
            os.getpid(), image.size[0], image.size[1]))

if __name__ == '__main__':
    queue = mp.Queue()
    finished = mp.Event()
    capture_proc = mp.Process(target=do_capture, args=(queue,
finished))
    processing_procs = [
        mp.Process(target=do_processing, args=(queue, finished))
        for i in range(4)
    ]

    for proc in processing_procs:
        proc.start()
    capture_proc.start()

    for proc in processing_procs:
        proc.join()
    capture_proc.join()

```

5.17 VLC 不能播放 MJPEG 视频

MJPEG 是一种特别不明确的格式（请参阅“Motion JPEG”），这会导致声称生成 MJPEG 文件的软件与播放 MJPEG 文件的软件之间存在兼容性问题。这是一种这样的情况：树莓派相机固件生成一个 MJPEG 文件，该文件仅由连接的 JPEG 组成；这在其他设备和网络摄像头相当常见，并且是一种非常简单的格式，使解析特别容易（请参阅 Web 流媒体示例）。

不幸的是，VLC 不会将其识别为有效的 MJPEG 文件：它认为它是单个 JPEG 图像并且不会读取文件的其余部分（在没有任何其他信息的情况下这也是一个合理的解释）。值得庆幸的是，可以提供额外的命令行开关来暗示文件中还有更多内容可供阅读：

```
$ vlc --demux=mjpeg --mjpeg-fps=30 my_recording.mjpeg
```

六、相机硬件

本章概述了相机在各种条件下的工作方式，并介绍了 picamera 使用的软件界面。

6.1 操作理论

我收到的许多关于 picamera 的问题都是基于对相机工作原理的误解。本章试图纠正这些误解，并为读者提供有关相机操作的基本说明。本章特意采用了简单模型，首先展示了一个技术上不准确但有用的相机操作模型，然后将其提炼得更接近真相。

6.1.1 误解1

树莓派的摄像头模组基本上就是手机摄像头模组。手机数码相机在一些方面不同于更大、更昂贵的相机 (DSLR)。其中最重要的是，对于了解树莓派的相机，许多移动相机（包括树莓派的相机模块）使用滚动快门来捕获图像。当相机需要捕获图像时，它会一次一行地从传感器中读取像素，而不是一次捕获所有像素值。

事实上，数码单反相机上的“全局快门”通常也一次读取一行像素。主要区别在于 DSLR 将具有覆盖传感器的物理快门。因此，在数码单反相机中，捕捉图像的过程是打开快门，让传感器“观察”场景，关闭快门，然后从传感器读取每一行。

因此，“捕获图像”的概念有点误导，因为我们实际上的意思是“依次从传感器读取每一行并将它们组合成图像”。

6.1.2 误解2

相机在捕捉帧之前实际上处于空闲状态的想法也具有误导性。不要将相机视为静止图像相机。把它想象成一个摄像机。特别是，一旦初始化，它就会不断地将帧（或帧行）沿着带状电缆向下传输到树莓派进行处理。

相机可能看起来空闲，您的脚本可能对相机没有任何作用，但后台仍有许多任务在进行（自动增益控制、曝光时间、白平衡以及我们稍后将介绍的其他几个任务）。

这种后台处理是前几章中看到的大多数 picamera 示例脚本在初始化相机后包含 `sleep(2)` 行的原因。`sleep(2)` 语句将您的脚本暂停几秒钟。在此暂停期间，相机的固件不断从相机接收帧行并调整传感器的增益和曝光时间，使帧看起来“正常”（不会曝光过度或曝光不足等）。

因此，当我们要求相机“捕捉一帧”时，我们真正要求的是相机给我们它组装的下一个完整帧，而不是将其用于增益和曝光然后将其丢弃（就像背景中经常发生的那样）。

6.1.3 曝光时间

相机传感器实际检测到什么？它检测光子计数；击中传感器元件的光子越多，这些元件的计数器增加就越多。由于树莓派相机没有物理快门（与 DSLR 不同），无法防止光线落在元素上并增加计数。实际上我们只能对传感器进行两种操作：重置一行元素，或者读取一行元素。

要了解典型的帧捕获，让我们来看看使用假设的相机传感器捕获几帧数据的过程，该传感器只有 8×8 像素且没有拜耳过滤器。传感器处于强光下，但由于它刚刚初始化，所有元素都以 0 计数开始。传感器的元素显示在左侧，我们将读取值的帧缓冲区正确的打开：

[illegible]

第一行数据被重置（在这种情况下，不会改变任何传感器元件的状态）。在重置第一行数据的同时，光线仍然被探测器其他单元接收，因此它们增加 1:

[illegible]

第二行数据被重置（这次，一些传感器状态发生了变化），所有其他元素递增 1。由于还没有读取任何内容，因为在读取第一行之前，需要为第一行“看到”足够的光线留出延迟：

Sensor elements								—>	Frame 1							
1	1	1	1	1	1	1	1									
0	0	0	0	0	0	0	0	Rst								
2	2	2	2	2	2	2	2									
2	2	2	2	2	2	2	2									
2	2	2	2	2	2	2	2									
2	2	2	2	2	2	2	2									
2	2	2	2	2	2	2	2									
2	2	2	2	2	2	2	2									

第三行数据被重置。同样，所有其他元素都增加 1：

Sensor elements								—>	Frame 1							
2	2	2	2	2	2	2	2									
1	1	1	1	1	1	1	1									
0	0	0	0	0	0	0	0	Rst								
3	3	3	3	3	3	3	3									
3	3	3	3	3	3	3	3									
3	3	3	3	3	3	3	3									
3	3	3	3	3	3	3	3									
3	3	3	3	3	3	3	3									

现在相机开始读取和重置。读取第一行，重置第四行：

Sensor elements								—>	Frame 1							
3	3	3	3	3	3	3	3		3	3	3	3	3	3	3	3
2	2	2	2	2	2	2	2									
1	1	1	1	1	1	1	1									
0	0	0	0	0	0	0	0	Rst								
4	4	4	4	4	4	4	4									
4	4	4	4	4	4	4	4									
4	4	4	4	4	4	4	4									
4	4	4	4	4	4	4	4									

读取第二行，同时重置第五行：

Sensor elements								—>	Frame 1							
4	4	4	4	4	4	4	4		3	3	3	3	3	3	3	3

Sensor elements								—>	Frame 1							
3	3	3	3	3	3	3	3	—>	3	3	3	3	3	3	3	3
2	2	2	2	2	2	2	2									
1	1	1	1	1	1	1	1									
0	0	0	0	0	0	0	0	Rst								
5	5	5	5	5	5	5	5									
5	5	5	5	5	5	5	5									
5	5	5	5	5	5	5	5									

按照逻辑，接下来可以快进到最后一行。

Sensor elements								—>	Frame 1							
7	7	7	7	7	7	7	7		3	3	3	3	3	3	3	3
6	6	6	6	6	6	6	6		3	3	3	3	3	3	3	3
5	5	5	5	5	5	5	5		3	3	3	3	3	3	3	3
4	4	4	4	4	4	4	4		3	3	3	3	3	3	3	3
3	3	3	3	3	3	3	3	—>	3	3	3	3	3	3	3	3
2	2	2	2	2	2	2	2									
1	1	1	1	1	1	1	1									
0	0	0	0	0	0	0	0	Rst								

此时，相机可以再次开始重置第一行，同时继续从传感器读取剩余的行：

Sensor elements								—>	Frame 1							
0	0	0	0	0	0	0	0	Rst	3	3	3	3	3	3	3	3
7	7	7	7	7	7	7	7		3	3	3	3	3	3	3	3
6	6	6	6	6	6	6	6		3	3	3	3	3	3	3	3
5	5	5	5	5	5	5	5		3	3	3	3	3	3	3	3
4	4	4	4	4	4	4	4		3	3	3	3	3	3	3	3
3	3	3	3	3	3	3	3	—>	3	3	3	3	3	3	3	3
2	2	2	2	2	2	2	2									
1	1	1	1	1	1	1	1									

快进到最后一行已被读取的状态，第一帧现在已经完成：

Sensor elements								—>	Frame 1							
2	2	2	2	2	2	2	2		3	3	3	3	3	3	3	3
1	1	1	1	1	1	1	1		3	3	3	3	3	3	3	3
0	0	0	0	0	0	0	0	Rst	3	3	3	3	3	3	3	3
7	7	7	7	7	7	7	7		3	3	3	3	3	3	3	3

Sensor elements								—>	Frame 1							
6	6	6	6	6	6	6	6		3	3	3	3	3	3	3	3
5	5	5	5	5	5	5	5		3	3	3	3	3	3	3	3
4	4	4	4	4	4	4	4		3	3	3	3	3	3	3	3
3	3	3	3	3	3	3	3	—>	3	3	3	3	3	3	3	3

在这个阶段，第 1 帧将被发送后进行处理，第 2 帧将被读入一个新的缓冲区：

Sensor elements								—>	Frame 1							
3	3	3	3	3	3	3	3	—>	3	3	3	3	3	3	3	3
2	2	2	2	2	2	2	2									
1	1	1	1	1	1	1	1									
0	0	0	0	0	0	0	0	Rst								
7	7	7	7	7	7	7	7									
6	6	6	6	6	6	6	6									
5	5	5	5	5	5	5	5									
4	4	4	4	4	4	4	4									

从上面的例子可以清楚地看出，我们可以通过改变重置一行和读取它之间的延迟来控制帧的曝光时间（这对于这个流程，重置和读取实际上不是同时发生的，但它们是同步的）。

6.1.3.1 最短曝光时间

最短曝光时间定义为读出一行元素花费的最短时间。例如，如果我们假设的传感器上有 500 行，并且读取每一行至少需要 20ns，那么读取完整帧至少需要 $500 \times 20 \text{ ns} = 10\text{ms}$ 。这是我们假设的传感器的最短曝光时间。

6.1.3.2 最大帧率由最小曝光时间决定

帧率是相机每秒可以捕捉的帧数。根据捕获一帧所需的时间，即曝光时间，我们只能在特定时间内捕获这么多帧。

例如：采样时间如果为 10ms，那么每秒钟的采样帧率就不可能超过 $\frac{1\text{s}}{10\text{ms}} = \frac{1\text{s}}{0.01\text{s}} = 100$ 帧图像，因此，我们假设 500 行传感器的最大帧速率为 100fps。

从方程中可以看出： $\frac{1\text{s}}{\min \text{ exposure time in s}} = \max \text{ framerate in fps}$ ，曝光时间和帧速率成反比关系，最小曝光时间越小，最大帧速率越大，反之亦然。

6.1.3.3 最大曝光时间由最小帧率决定

为了最大化曝光时间，我们需要每秒捕获尽可能少的帧，即我们需要非常低的帧率。因此，最大曝光时间由相机的最小帧率决定。最小帧率很大程度上取决于传感器读取行的速度（在硬件级别，这取决于用于保存行读出时间等内容的寄存器的大小）。

这可以用单词方程表示： $\frac{1\text{s}}{\min\ \text{framerate}\ \text{in}\ \text{fps}} = \max\ \text{exposure}\ \text{time}$

如果我们假设传感器的最小帧率为 $\frac{1}{2}\text{fps}$ ，则最大曝光时间为 $\frac{1\text{s}}{1/2} = 2\text{s}$ 。

6.1.3.4 曝光时间受当前帧率限制

相机的帧率设置限制了给定framerate的最大曝光时间。例如，如果我们将帧率设置为 30fps，那么我们花费的时间不能超过 $\frac{1\text{s}}{30} = 33\frac{1}{3}\text{ms}$ 捕获任何给定的帧。

因此，exposure_speed属性报告最后处理帧的曝光时间（实际上是传感器线读出时间的倍数）受相机帧framerate的限制。

Note

使用framerate_delta完成的微小帧率调整是通过在帧末尾读取额外的“虚拟”行来实现的。即阅读一行，然后将其丢弃。

6.1.4 传感器增益

除了读出时间之外，影响传感器元件计数的另一个重要因素是传感器的增益。具体来说，是由analog_gain属性给出的增益（相应的digital_gain只是后处理，我们将在后面介绍）。然而，有一个明显的问题：如果我们处理数字光子计数，这种增益如何“模拟”？

是时候揭开第一个谎言了：传感器元件不是简单的数字计数器，而是随着更多光子撞击它们而积累电荷的模拟元件。模拟增益会影响电荷的累积方式。模数转换器 (ADC) 用于在线路读出期间将模拟电荷转换为数字值（实际上 ADC 的速度是最小线路读出时间的很大一部分）。

Note

相机传感器也往往具有非感应像素（被光覆盖的元素）的边界。这些用于确定什么水平的电荷代表“光学黑色”。

相机的元件会受到热量的影响（毕竟，热辐射只是接近可见光部分的电磁波谱的一部分）。如果没有非感应像素，您将在不同的环境温度下获得不同的暗电流。

模拟增益不能在 `picamera` 中直接控制，但可以使用各种属性来“影响”它。

- 将 `Exposure_mode` 设置为 "off" 会将模拟（和数字）增益锁定在它们的当前值，并且根本不允许它们进行调整，无论场景发生什么，也无无论其他相机属性可能会被调整。
- 将 `exposure_mode` 设置为 'off' 以外的值允许增益根据选择的自动曝光模式“浮动”（改变）。在可能的情况下，相机固件更喜欢调整模拟增益而不是数字增益，因为增加数字增益会产生更多的噪音。为不同的自动曝光模式所做的调整的一些示例包括：
 - "运动"通过优先增加增益而不是曝光时间（即行读出时间）来减少运动模糊。
 - "夜间"旨在作为静止模式，因此它允许很长的曝光时间，同时试图保持低增益。
- `iso` 属性有效地代表了另一组具有特定增益的自动曝光模式：
 - 对于 V1 相机模块，ISO 100 尝试使用 1.0 的整体增益。ISO 200 尝试使用 2.0 的整体增益，依此类推。
 - 使用 V2 相机模块时，ISO 100 可产生 ~1.84 的整体增益。ISO 60 的整体增益为 1.0，ISO 800 为 14.72（V2 相机模块根据 ISO 胶片速度标准进行校准）。

因此，人们可能会认为 `iso` 提供了一种修复增益的方法，但这并不完全正确：`exposure_mode` 设置优先（将曝光模式设置为 "off" 将修复增益，无论以后 ISO 是什么 设置，并且某些曝光模式（如 "spotlight"）也会覆盖 ISO 调整的增益）。

6.1.5 系统分工

熟悉操作系统理论的读者可能会质疑像 Linux 这样的非实时操作系统 (non-RTOS) 怎么可能从传感器读取行？毕竟，要确保在完全相同的时间内读取每一行（以确保整个帧的持续曝光）需要极其精确的计时，这在非 RTOS 中是无法实现的。

是时候揭开第二个谎言了：CPU并不会主动从传感器“读取”行数据。相反，传感器通过其寄存器配置每行的时间和要读取的行数。一旦启动，传感器只需读取行，以配置的速度将数据推送到树莓派。

这说明了每一行的读出时间是如何保持不变的，但它仍然没有回答我们如何保证 Linux 确实在侦听并准备好接受每一行数据的问题？答案很简单，Linux 没有。CPU 不直接与相机通话。事实上，运行 Linux 时，相机数据的处理根本没有发生在 CPU 上。相反，它是通过操作系统 (VCOS) 实时运行在 的树莓派的 GPU (VideoCore IV) 上的。

Note

这是另一个谎言：VCOS 实际上是在 GPU 上运行的 RTOS（在撰写本文时为 ThreadX）之上的抽象层。然而，考虑到 RTOS 在过去发生了变化（因此是抽象层），并且用户无论如何都不会直接与之交互，将 GPU 视为运行称为 VCOS 的东西（无需考虑太多）可能更简单。

下图说明了 BCM2835 片上系统 (SoC) 由运行 Linux 的 ARM Cortex CPU（在其下运行使用 picamera 的 myscript.py）和运行 VCOS 的 VideoCore IV GPU 组成。VideoCore 主机接口 (VCHI) 是一个消息传递系统，用于允许这两个组件之间的通信。可用 RAM 在两个组件之间拆分（128Mb 是使用相机时典型的 GPU 内存拆分）。最后，摄像头模块显示在 SoC 上方。它通过 CSI-2 接口（提供 2Gbps 的带宽）连接到 SoC。

使用场景描述如下：

1. 摄像头的传感器已配置好，并不断通过 CSI-2 接口将帧线传输到 GPU。
2. GPU 正在从这些行组装完整的帧缓冲区并对这些缓冲区执行后处理（我们将在下一节中进一步详细介绍这部分）。
3. 同时，在 CPU 上，myscript.py 使用 picamera 进行捕获调用。
4. picamera 库反过来使用 MMAL API 来制定这个请求（实际上这里有很多 MMAL 调用，但为了简单起见，我们用一个箭头表示所有这些）。
5. MMAL API 通过 VCHI 发送请求帧捕获的消息（同样，实际上有比单个消息更多的活动）。
6. 作为响应，GPU 启动从其 RAM 部分到 CPU 部分的下一个完整帧的 DMA 传输。
7. 最后，GPU 通过 VCHI 发回一条消息，表示捕获已完成。
8. 这会导致 MMAL 线程在 picamera 库中触发回调，进而检索帧（实际上，这需要更多的 MMAL 和 VCHI 活动）。
9. 最后，picamera 对myscript.py提供的输出对象调用 write。

6.1.6 后台进程

上面的部分中简要提到了一些 GPU 处理（增益控制、曝光时间、白平衡、帧编码等）。是时候揭开最后的谎言了：GPU 不是一个独立器件，而是一个离散组件，它由许多组件组成，每个组件都在相机的操作中发挥作用。

下图更准确地描述了 BCM2835 SoC 的 GPU 端。从这里我们第一次看到了帧处理“管道”以及为什么这样称呼它。在图中，正在录制 H264 视频。数据通过的组件如下：

1. 一旦相机开始工作，会进行一些小的处理。具体来说，翻转（水平和垂直）、跳线和像素合并是在传感器的寄存器上配置的。像素合并实际上发生在传感器本身，在 ADC 之前，以提高信噪比。请参见 `hflip`、`vflip` 和 `sensor_mode`。
2. 如前所述，帧线通过 CSI-2 接口传输到 GPU。在那里，它被 Unicam 组件接收，该组件将行数据写入 RAM。
3. 接下来，GPU 的图像信号处理器（ISP）对帧数据执行几个后处理步骤。

包括（按顺序）：

- 转置：如果已请求任何旋转，则将输入转置以旋转图像（旋转始终通过转置和翻转的某种组合实现）。
- 暗电流补偿：使用非光感元件（通常在覆盖的边框中）来确定代表“光学黑”的电荷水平。
- 镜头阴影：相机固件包括一个表格，用于校正标准模块镜头的色度失真。这就是包含不同镜头的第三方模块可能会在整个框架中显示不均匀颜色的原因之一。
- 白平衡：应用红色和蓝色增益来校正色彩平衡。请参阅 `awb_gains` 和 `awb_mode`。
- 数字增益：如上所述，这是将增益应用于拜耳值的直接后处理步骤。见 `digital_gain`。
- 拜耳降噪：这是一种在帧数据上运行的降噪算法，同时它仍为拜耳格式。
- 去马赛克：将帧数据从拜耳格式转换为 YUV420，这是管道其余部分使用的格式。
- YUV 去噪：另一种降噪算法，这次使用的是 YUV420 格式的帧。请参阅 `image_denoise` 和 `video_denoise`。
- 锐化：一种增强图像边缘的算法。见锐度。

- 颜色处理：实施亮度、对比度和饱和度调整。
- 失真：校正由相机镜头引入的失真。目前这个阶段没有任何作用，因为库存镜头不是鱼镜头；如果未来的传感器需要它，它作为一个选项存在。
- 调整大小：此时，将帧调整为请求的输出分辨率（所有先前阶段都已在“完整”帧数据上以传感器配置为产生的任何分辨率执行）。见分辨率。

其中一些步骤可以直接控制（例如亮度、降噪），其他步骤只能受到影响（例如模拟和数字增益），其余的根本不是用户可配置的（例如去马赛克和镜头阴影）。

在这一点上，框架实际上是“完整的”。

4. 如果您正在生成“未编码”数据输出（YUV、RGB 等），管道将在此时结束，帧数据将通过 DMA 复制到 CPU。ISP 可能用于转换为 RGB，但仅此而已。
5. 如果您正在生成编码输出（H264、MJPEG、MPEG2 等），下一步是编码块之一，在本例中为 H264 块。编码块是专门设计用于生成特定编码的专用硬件。例如，JPEG 模块将包括用于执行大量并行离散余弦变换 (DCT) 的硬件，而 H264 模块将包括用于执行运动估计的硬件。
6. 编码后，输出将通过 DMA 复制到 CPU。
7. 协调这些组件的是 VPU，它是运行 VCOS (ThreadX) 的 GPU 中的通用组件。VPU 配置和控制其他组件以响应来自 VCHI 的消息。目前最完整的 VPU 文档可从 `videocoreiv` 存储库中获得。

6.1.7 反馈循环

有几个反馈循环在上述管道内运行。当 `Exposure_mode` 不是 "off" 时，自动增益控制 (AGC) 会从每个帧（在 ISP 中的去马赛克阶段之前）收集统计信息。它会调整模拟和数字增益，以及尝试将后续帧的曝光时间（线读出时间）设定为目标 Y（亮度）值。

同样，当 `awb_mode` 不是 "off" 时，自动白平衡 (AWB) 会从帧中收集统计信息（同样在去马赛克之前）。通常，AWB 分析只发生在每 3 个流帧中的 1 个，因为它的计算成本很高。它调整红色和蓝色增益 (`awb_gains`)，试图将后续帧推向预期的色彩平衡。

在白天，您可以很容易地观察 AGC 效果。确保相机模块指向明亮的物体，如天空或窗外的景色，并查询相机的模拟增益和曝光时间：

```
>>> camera = PiCamera()
>>> camera.start_preview(alpha=192)
>>> float(camera.analog_gain)
1.0
>>> camera.exposure_speed
3318
```

如果将 iso 设置为 800 来强制相机使用更高的增益。如果运行预览，场景中的差异很小。但是，如果您随后查询曝光时间，您会发现固件已大幅减少曝光时间以补偿更高的传感器增益：

```
>>> camera.iso = 800
>>> camera.exposure_speed
198
```

可以使用 shutter_speed 属性强制延长曝光时间，此时场景将变得非常亮（因为增益和曝光时间现在都是固定的）。如果通过将 iso 设置回自动 (0)，让增益再次浮动，应该会发现增益相应降低并且场景图像或多或少恢复正常：

```
>>> camera.shutter_speed = 4000
>>> camera.exposure_speed
3998
>>> camera.iso = 0
>>> float(camera.analog_gain)
1.0
```

相机的 AGC 循环尝试在 ISO、快门速度等设置的约束范围内生成具有目标 Y（亮度）值（或多个值）的场景。目标 Y' 值可以通过 Exposure_compensation 属性进行调整，该属性以 f-stop 的 1/6 为增量进行测量。因此，如果在曝光时间固定的情况下，您将相机瞄准的亮度增加了几档，然后等待几秒钟，您应该会发现增益相应增加：

```
>>> camera.exposure_compensation = 12
>>> float(camera.analog_gain)
1.48046875
```

如果让曝光时间再次浮动（通过将 shutter_speed 设置回 0），然后等待几秒钟，您应该会发现模拟增益降低回 1.0，但曝光时间增加仍旧会引起场景的过度曝光：


```
>>> camera.shutter_speed = 0
>>> float(camera.analog_gain)
1.0
>>> camera.exposure_speed
4244
```

6.2 传感器模式

树莓派相机模块具有多种采样模式，可用于将数据输出到 GPU。在 V1 模块上，这些如下所示：

#	Resolution	Aspect Ratio	Framerates	Video	Image	FOV	Binning
1	1920×1080	16:9	$1 < \text{fps} \leq 30$	$\times$		Partial	None
2	2592×1944	4:3	$1 < \text{fps} \leq 15$	$\times$	$\times$	Full	None
3	2592×1944	4:3	$\frac{1}{6} \leq \text{fps} \leq 1$	$\times$	$\times$	Full	None
4	1296×972	4:3	$1 < \text{fps} \leq 42$	$\times$		Full	2×2
5	1296×730	16:9	$1 < \text{fps} \leq 49$	$\times$		Full	2×2
6	640×480	4:3	$42 < \text{fps} \leq 60$	$\times$		Full	4×4
7	640×480	4:3	$60 < \text{fps} \leq 90$	$\times$		Full	4×4

在 V2 模块上，表现为：

#	Resolution	Aspect Ratio	Framerates	Video	Image	FOV	Binning
1	1920×1080	16:9	$1 < \text{fps} \leq 30$	$\times$		Partial	None
2	3280×2464	4:3	$\frac{1}{10} < \text{fps} \leq 15$	$\times$	$\times$	Full	None
3	3280×2464	4:3	$\frac{1}{10} \leq \text{fps} \leq 15$	$\times$	$\times$	Full	None
4	1640×1232	4:3	$\frac{1}{10} < \text{fps} \leq 40$	$\times$		Full	2×2
5	1640×922	16:9	$\frac{1}{10} < \text{fps} \leq 40$	$\times$		Full	2×2
6	1280×720	16:9	$40 < \text{fps} \leq 90$	$\times$		Partial	2×2
7	640×480	4:3	$40 < \text{fps} \leq 90$	$\times$		Partial	2×2

Note

以上并不是全部的可能输出分辨率或帧率。这些只是传感器可以直接输出到 GPU 的一组分辨率和帧率。GPU 的 ISP 块在合理范围内将调整为任何请求的分辨率。请继续阅读有关模式选择的详细信息。

Note

V2 模块上的传感器模式 3 似乎是重复的，但这是故意的。V2 模块的传感器模式旨在模拟与 V1 模块最接近的等效传感器模式。V1 模块上的长时间曝光需要单独的传感器模式；但在 V2 模块上不是必需的。

从相机传感器的整个区域（V1 相机为 2592x1944 像素，V2 相机为 3280x2464）捕获全视野 (FoV) 模式。从传感器中心捕获部分 FoV 的模式。FoV 限制和分箱的组合用于实现所需的分辨率。

下图说明了 V1 相机的完整视野和部分视野之间的区别：

V2 相机的各种视野如下图所示：

可以使用 PiCamera 构造函数中的 `sensor_mode` 参数强制传感器的模式（使用上表中 # 列中的值之一）。此参数默认为 0，表示应根据请求的分辨率和帧率自动选择模式。选择哪种传感器模式的规则如下：

- 采样模式必须是可接受的。所有模式都可用于视频录制，或从视频端口捕获图像（即当 `use_video_port` 在调用各种捕获方法时为 `True` 时）。当 `use_video_port` 为 `False` 时的图像捕获必须使用图像模式（其中只有两种存在，都具有最大分辨率）。
- `resolution` 越接近模式的分辨率越好，但从较高的传感器分辨率降到较低的输出分辨率比从较低的传感器分辨率升频更可取。
- `framerate` 应该在传感器模式的范围内。
- `resolution` 的纵横比与模式分辨率越接近越好。尝试使用 4:3 或 16:9（这是上表中的模式直接支持的唯一比率）以外的纵横比设置分辨率将选择最大化结果视野 (FoV) 的模式。

下面给出了几个例子来阐明这个操作（以 V1 相机模块为例）：

- 如果您将分辨率设置为 1024x768（4:3 纵横比），并将帧速率设置为低于 42fps，则将选择 1296x972 模式 (4)，并且 GPU 会将结果缩小到 1024x768。
- 如果您将分辨率设置为 1280x720（16:9 宽屏纵横比），并将帧速率设置为低于 49fps，则 1296x730 模式 (5) 将被选择并适当缩小。

- 将分辨率设置为 1920x1080 并将帧速率设置为 30fps，该模式超过了 1296x730 和 1296x972 模式的分辨率（即它们需要放大），因此选择了 1920x1080 模式 (1)，尽管它需要降低 FoV。
- 800x600 的分辨率和 60fps 的帧率将选择 640x480 60fps 模式，即使它需要升级，因为在这种情况下算法认为帧率优先。
- 在执行采样时，任何不使用视频端口捕获图像的尝试都将（临时）选择 2592x1944 模式（这就是导致在捕获静止图像的同时运行预览时有时会看到闪烁的原因）。

6.3 硬件限制

采用 GPU 硬件执行所有图像和视频处理都有额外限制：

MJPEG 记录的最大分辨率部分取决于 GPU 内存。如果您在高分辨率 MJPEG 记录时遇到“资源不足”错误，请尝试增加 `/boot/config.txt` 中的 `gpu_mem`。

默认 H264 录制的最大水平分辨率为 1920（这是 GPU 中 H264 块的限制）。任何以更高水平分辨率录制 H264 视频的尝试都将失败。

在低帧率 (<1fps) 下运行时，V2 相机的最大分辨率可能需要额外的 GPU 内存。如果在尝试使用 V2 模块进行长时间曝光捕获时遇到“资源不足”错误，请增加 `/boot/config.txt` 中的 `gpu_mem`。

V2 相机的最大分辨率也会导致预览出现问题。目前，picamera 以与捕获相同的分辨率运行预览（相当于 `raspistill` 中的 `-fp`）。您可能需要在 `/boot/config.txt` 中增加 `gpu_mem` 以实现使用 V2 相机模块的全分辨率操作，或者将预览配置为使用比相机本身更低的分辨率。

相机的最大帧率取决于几个因素。通过超频，V2 模块可以达到 120fps，但 90fps 是支持的最大帧率。

目前在 V1 摄像头模组上最大曝光时间为 6 秒，在 V2 摄像头模组上为 10 秒。请记住，曝光时间受帧率限制，因此您需要在设置 `shutter_speed` 之前设置极慢的帧率。

6.4 MMAL

picamera 下的 MMAL 层为在 GPU 上运行的相机固件提供了一个大大简化的接口。从概念上讲，它为相机提供了三个“端口”：静止端口、视频端口和预览端口。以下部分描述了 picamera 如何使用这些端口以及它们如何影响相机的行为。

6.4.1 静止端口

首先，静止端口用于捕获图像时，它强制相机的模式为两种支持的静止模式之一（请参阅传感器模式），以便使用传感器的整个区域捕获图像。它还对捕获的图像使用了强大的降噪算法，使它们看起来质量更高。

当`use_video_port`参数为`False`（默认情况）时，各种`capture()`方法使用静态端口。

6.4.2 视频端口

视频端口更简单一些，因为它永远不会改变相机的模式。视频端口由`start_recording()`函数（用于录制视频）使用，并且当它们的`use_video_port`参数为`True`时，也由各种`capture()`函数调用。从视频端口捕获的图像往往具有“颗粒状”外观，比静态端口捕获的图像更类似于视频帧（这是由于静态端口使用了更强的降噪算法）。

6.4.3 预览端口

预览端口的操作与视频端口大致相同。预览端口始终连接到某种形式的输出，以确保自动增益算法可以运行。当构建`PiCamera`的实例时，预览端口最初连接到`PiNullSink`的实例。当`start_preview()`被调用时，这个空接收器被销毁并且预览端口连接到`PiPreviewRenderer`的一个实例。调用`stop_preview()`时会发生相反的情况。

6.4.4 管道

本节试图详细说明 `picamera` 函数由MMAL管道为响应各种方法调用而构造的内容。

固件提供了各种编码器，它们可以连接到静态和视频端口以产生输出（例如 JPEG 图像或 H.264 编码视频）。端口可以在任何给定时间连接一个编码器（如果端口未使用，则不连接任何编码器）。

编码器直接连接到静止端口。例如，当使用静态端口捕捉图片时，相机的状态概念上会在这些状态之间移动：

视频端口稍微复杂一些。为了允许通过视频端口同时进行视频录制和图像捕获，“分离器”组件通过 `picamera` 永久连接到视频端口，编码器依次连接到其四个输出端口之一（编号为 0、1、2，和 3）。因此，在录制视频时，摄像机的设置如下所示：

并且在录制的同时通过视频端口捕获图像时，摄像机的配置会经历以下状态：

当`resize`参数传递给上述方法之一时，会在相机端口和编码器之间放置一个调整器组件，导致输出在到达编码器之前调整大小。这对于视频录制特别有用，因为 H.264 编码器无法处理全分辨率输入（GPU 硬件只能处理高达 1920 像素的帧宽度）。因此，在执行全帧视频录制时，相机的设置如下所示：

最后，在执行未编码的捕获时不需要编码器。而是直接从相机的端口获取数据。然而，各种固件限制需要在管道中进行杂技来实现请求的编码。

例如，在旧固件中，相机的静止端口无法实现 RGB 输出（由于缓冲区大小不适配）。但是，它们可以配置为 YUV 输出，因此在这种情况下，picamera 为 YUV 输出配置静态端口，附加为调整器（配置为具有相同的输入和输出分辨率），然后将调整器的输出配置为 RGBA（调整器不支持 RGB 出于某种原因）。然后它运行捕获并从数据中去除冗余的 alpha 字节。

最近的固件（1.11版本）修复了缓冲区大小，因此使用这些 picamera 将简单地配置 RGB 输出的静止端口：

6.4.5 编码

MMAL 组件将端口以特定编码方式连接在一起，从而进行传递图像数据。通常，这是 YUV420 编码（这是管道的“首选”内部格式）。在极少数情况下，它是 RGB（RGB 是一种大而低效的格式）。然而，有时使用的另一种格式是“OPAQUE”编码。

“OPAQUE”是连接 MMAL 组件时使用的最有效的编码，因为它只是在后台传递指针而不是全帧数据（因此它根本不是真正的编码，但它被 MMAL 框架视为这样）。但是，并非所有 OPAQUE 编码都是等效的：

- 预览端口的 OPAQUE 编码包含单个图像。
- 视频端口的 OPAQUE 编码包含两个图像（用于各种编码器的运动估计）。
- 静止端口的 OPAQUE 编码包含空图像。

- JPEG 图像编码器接受静止端口的 OPAQUE 空图像。
- MJPEG 视频编码器不接受 OPAQUE 空图像，仅接受预览或视频端口提供的单图像和双图像变体。
- 旧固件中的 H.264 视频编码器仅接受双图像 OPAQUE 格式（尽管它将接受全帧 YUV 输入）。在较新的固件中，它现在也接受单个图像 OPAQUE 格式（大概是构建第二个图像本身用于运动估计）。
- 分离器接受单或双图像 OPAQUE 输入，但只输出单图像 OPAQUE 输入（或 YUV；在以后的固件中它还支持 RGB 或 BGR 输出）。
- VPU 调整器 (MMALResizer) 理论上接受 OPAQUE 输入（尽管作者在撰写本文时还没有设法让它工作）但只会产生 YUV、RGBA 和 BGRA 输出，而不是 RGB 或 BGR。
- ISP 调整器 (MMALISPResizer，目前未被 picamera 的高级 API 使用，但可从 mmalobj 层获得) 接受 OPAQUE 输入，并且会产生几乎所有未编码的输出（包括 YUV、RGB、BGR、RGBA 和 BGRA）但不包括 OPAQUE。

picamera 1.11 版本中引入的 mmalobj 层意识到这些 OPAQUE 编码差异，并尝试使用最有效的格式来配置组件之间的连接。但是，它不知道固件修订版，因此如果您正在通过这一层使用 MMAL 组件，请准备好进行一些修补以使您的管道正常工作。

请注意，上面的描述是 MMAL 对成像管道的极大简化。这与 GPU 的 ISP 级别实际发生的情况相去甚远（在前面的部分中进行了粗略描述）。但是，由于 MMAL 是支持 picamera 库（以及官方 `raspistill` 和 `raspivid` 应用程序）的 API，因此值得了解。

换句话说，通过使用 picamera，您正在通过（至少）两个抽象层，这些抽象层必然会掩盖（但希望简化）相机的“真实”操作。

七、开发环境

项目在Github的地址为：

<https://github.com/waveform80/picamera>

非常欢迎使用者来讨论错误、新功能，或者只是提出使用问题（例如，我发现这对于衡量常见问题解答中应该包含哪些问题很有用）。

对于任何希望破解该项目的人，我强烈建议通读 PiCamera 类的源代码，以掌握使用 mmalobj 层的方法。这是 picamera 1.11 中引入的一个层，以简化 libmmal（picamera、raspistill 和 raspivid 都依赖的底层库）的使用。

mmalobj 下面是 libmmal 头文件的 ctypes 翻译，但我希望大多数开发人员永远不需要直接处理这个问题（即：希望不再需要 C 的工作知识来破解 picamera）。

这个项目中还存在用于专门应用程序的各种类（PiCameraCircularIO、PiBayerArray 等）

即使您不想修改代码，我也很乐意听取人们的建议，了解您希望 API 的外观（即使代码本身不是特别 Pythonic，界面也应该是）！

7.1. 开发安装

如果您希望自己开发 picamera，最简单的方法是通过克隆 GitHub 仓库获取源代码，然后使用 Makefile 的“开发”目标，它将安装包作为克隆存储库的链接，允许本地开发（它还使用 Exuberant 的 ctags 实用程序构建了一个与 vim/emacs 一起使用的标签文件）。以下示例在虚拟 Python 环境中演示了此方法：

```
$ sudo apt-get install lsb-release build-essential git git-core \  
> exuberant-ctags virtualenvwrapper python-virtualenv python3-  
virtualenv \  
> python-dev python3-dev libjpeg8-dev zlib1g-dev libav-tools  
$ cd  
$ mkvirtualenv -p /usr/bin/python3 picamera  
$ workon picamera  
(picamera) $ git clone https://github.com/waveform80/picamera.git  
(picamera) $ cd picamera  
(picamera) $ make develop
```

如果要将最新的版本从 git 提取到您的克隆中并更新您的安装：

```
$ workon picamera
(picamera) $ cd ~/picamera
(picamera) $ git pull
(picamera) $ make develop
```

要删除您的安装，请销毁沙箱和克隆：

```
(picamera) $ deactivate
$ rmvirtualenv picamera
$ rm -fr ~/picamera
```

7.2. 构建文档

如果您希望构建文档，则需要更多的依赖项。Inkscape 可以用于将 SVG 转换为其他格式，Graphviz 用于渲染某些图表，TeX Live 来构建 PDF 输出。以下命令应安装所有必需的依赖项：

```
$ sudo apt-get install texlive-latex-recommended texlive-latex-extra \
texlive-fonts-recommended graphviz inkscape
```

安装这些后，您可以使用“doc”目标来构建文档：

```
$ workon picamera
(picamera) $ cd ~/picamera
(picamera) $ make doc
```

HTML 输出在文件夹 docs/_build/html，而 PDF 输出在文件夹 docs/_build/latex。

7.3. 开发测试

如果您希望运行 picamera 的开发测试，请按照上面开发安装中的说明进行操作，然后在沙箱中创建“测试”目标：

```
$ workon picamera
(picamera) $ cd ~/picamera
(picamera) $ make test
```


Warning

开发测试需要很长时间才能执行（在超频的树莓派3上至少需要 1 小时）。根据配置，它还可以锁定摄像头，并且需要重新启动树莓派才能重置，因此请确保您熟悉 SSH 或在需要重新启动时使用备用 TTY 访问命令行。

八、放弃的功能

picamera 库（在撰写本文时）已有近一年的历史，并且在这段时间内发展得非常迅速。有时，当向库添加新功能时，API 是显而易见的和自然的（例如 `start_recording()` 和 `stop_recording()`）。在其他时候，它不那么明显（例如未编码的捕获）并且我最初的尝试被证明不太理想。在这种情况下，我通过引入新方法或属性并弃用旧方法或属性，努力在不破坏向后兼容性的情况下改进 API。

这意味着，从 1.8 版开始，库周围有很多不推荐使用的功能，整理起来会很好，部分是为了简化库以进行调试，部分是为了为新用户简化它。为了消除我即将打破向后兼容性的任何担忧：我打算在弃用功能和删除它之间留出至少一年的时间，希望为人们提供足够的时间来迁移他们的脚本。

此外，为了区分向后不兼容的任何版本，我将根据语义版本控制增加主要版本号。换句话说，第一个删除当前已弃用功能的版本将是 2.0 版，而到 1.8 版的发布至少还需要一年时间。任何未来的 1.x 版本都将包含所有当前已弃用的功能。

当然，这仍然意味着人们需要一种方法来确定他们的脚本是否使用了 picamera 库中任何已弃用的功能。所有不推荐使用的功能都记录在案，并且该文档包括指向预期替换功能的指针（例如，参见 `raw_format`）。但是，Python 还提供了极好的方法来自动确定是否通过警告模块使用了任何已弃用的功能。

九、API - PiCamera类

十、API - 流

十一、API -

十二、API -

十三、API -

十四、API -

十五、 API -

十六、 API -

十七、更新日志

十八、 许可